

Artificial
Intelligence



ХИЙМЭЛ ОЮУН УХААН – J.CS712

Боловсруулсан багш:

Др., Дэд профессор Д. Золзаяа

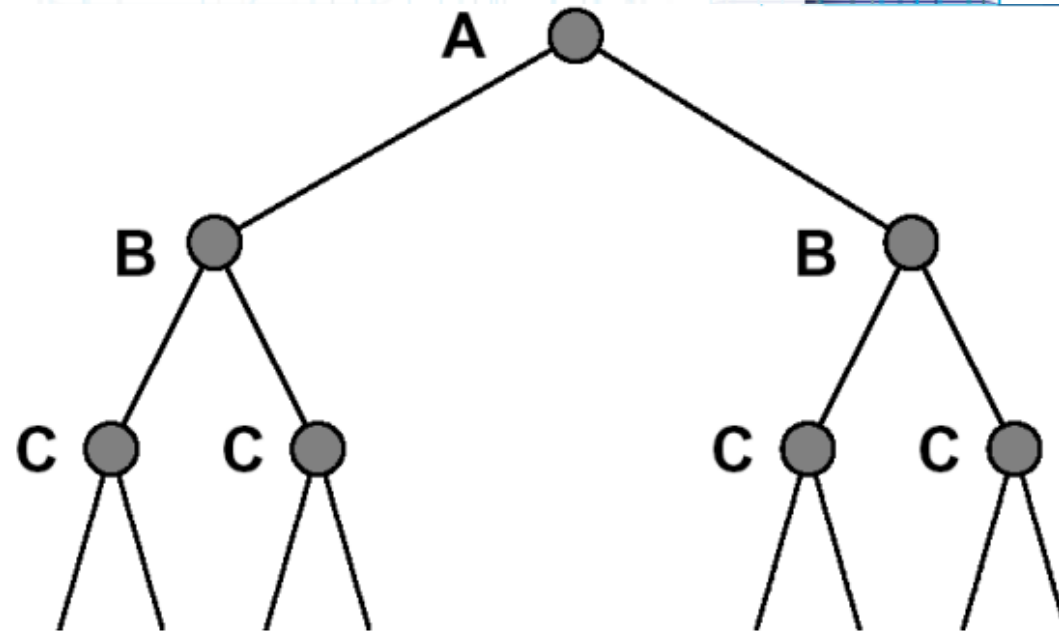
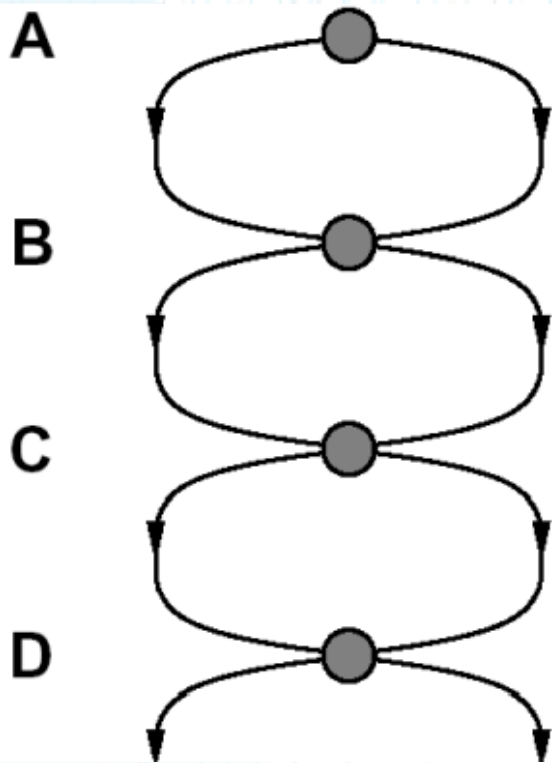
Мэйл хаяг: zolzaya@must.edu.mn

КУ-ны салбар

ШУТИС МХТС, 2021

Давхардсан төлөвөөс зайлсхийх

- Давхардлыг шалгахгүй бол шугаман асуудал ч экспоненциал болж болно.



Давхардсан төлөвөөс зайлсхийх

- Давхардлыг шалгах үндсэн гурван арга
 - Ирсэн төлөв рүүгээ буцан орохгүй байх.
Ө.х
 - Ирсэн төлөв рүүгээ буцан орохгүй байх.
Ө.х
 - Циклдсэн зам үүсгэхгүй байх
- Үүсгэсэн төлөвүүдийн мэдээллийг хадгалж шинээр үүсгэж байгаа төлөвүүдийг тэдэнтэй харьцуулан шалгах. (hash table)

Хайлтын асуудлууд

- Хайлтын асуудлууд нь агентыг байгаль орчинг бүрэн хянах боломжтой гэж үзсэн.
- Энэ таамаглал нь үргэлж үндэслэлгүй байдаг.
 - Стохастик орчин (цаг агаар, замын хөдөлгөөний осол)
 - Агентын ашиг сонирхолтой зөрчилдөж байгаа бусад агентууд

Хайлтын асуудлууд

- Эдгээр тохиолдлуудад бид агентуудын хяналтад байдаггүй төлвийн өөрчлөлтийг зохицуулахаар хайлтыг тодорхойлох хэрэгтэй.
- Нэг хэлбэр нь тоглоомын хайлтын модыг ашиглах
- Агент болон бусад агентууд
- Бусад агентууд ашгаа их байхаар ажилладаг.
 - Энэ нь таны ашигт эерэгээр нөлөөлөхгүй байж болно.

Хоёр тоглогчтой Zero-sum тоглоом

- Zero sum тоглоом нь илүү өрсөлдөөнтэй байдаг.
 - Нэг нь хожиж нөгөө нь хожигддог.
 - Таны хийх гэж буй үйлдэл өрсөлдөгчийн хэрхэн үйлдэл хийдэг вэ (эсвэл тэд хэрхэн ажиллахаа боддог вэ) гэдгээс хамаарна.
- Илүү ерөнхий зориулалтай тоглоомууд нь хамтын тоглолттой байдаг.
 - Зарим үр дүнг аль аль нь хоёулаа чухалчилдаг.

Хоёр тоглогчтой Zero-sum ТОГЛООМ

- Хэрхэн тоглох нь өрсөлдөгчөө хэрхэн тоглох гэж байгаагаас хамаарна; Гэхдээ өрсөлдөгч чинь таны хэрхэн тоглох талаар бодож байгаагаас хамаарна; Тэгэхээр чи яаж тоглох гэж байгаагаа бодож байгаагаасаа шалтгаална. Гэхдээ тэд хэрхэн тоглох гэж байгаа гэж бодож байгаа гэж боддогоо бодолцох хэрэгтэй. ...

Zero sum тоглоомын жишээнүүд

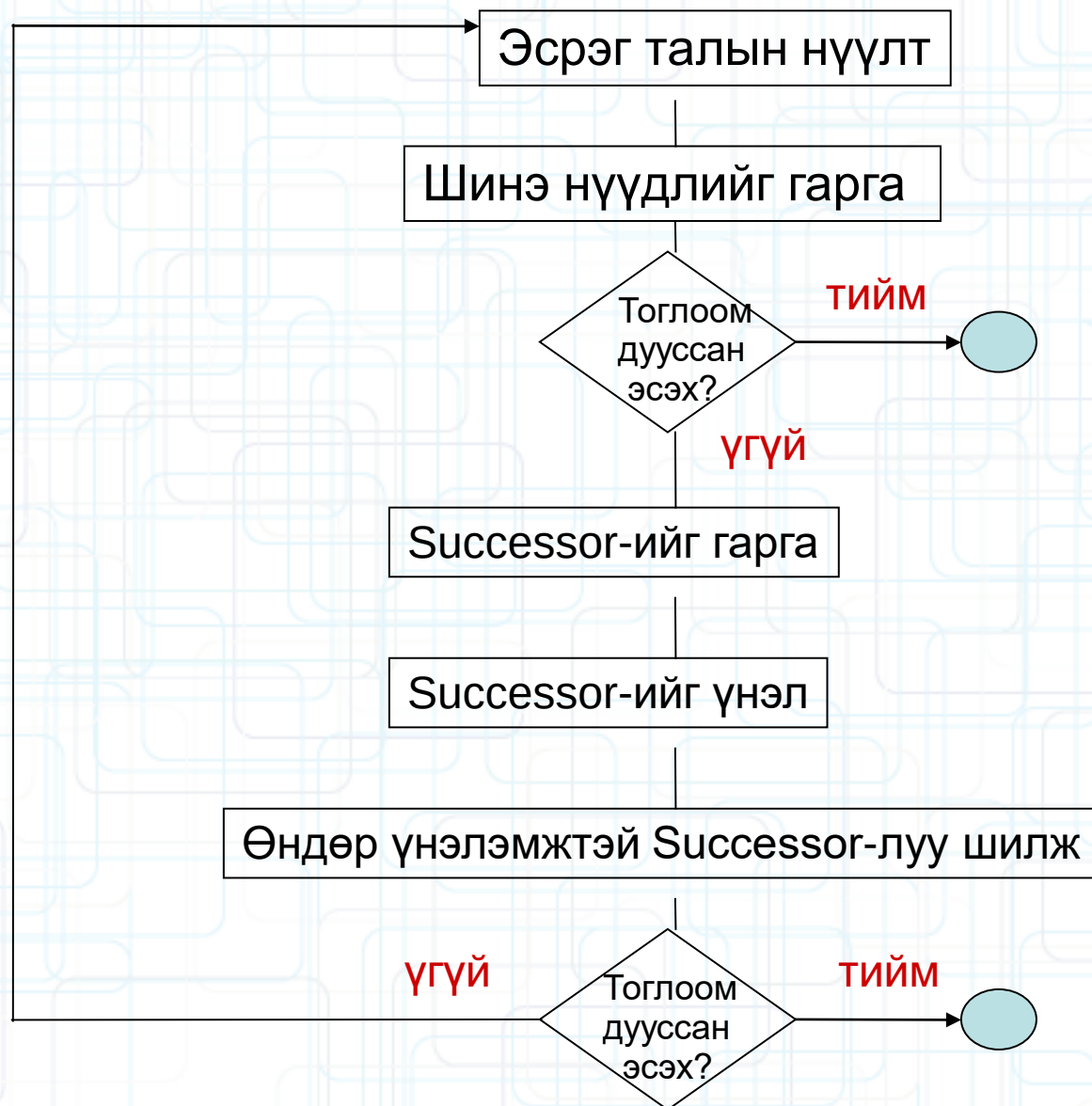
- Chess
- Checkers
- Tic-tac-toe
- Backgammon
- Go
- Doom
- “find the last parking space”

Хайч, чулуу, цаас тоглоом

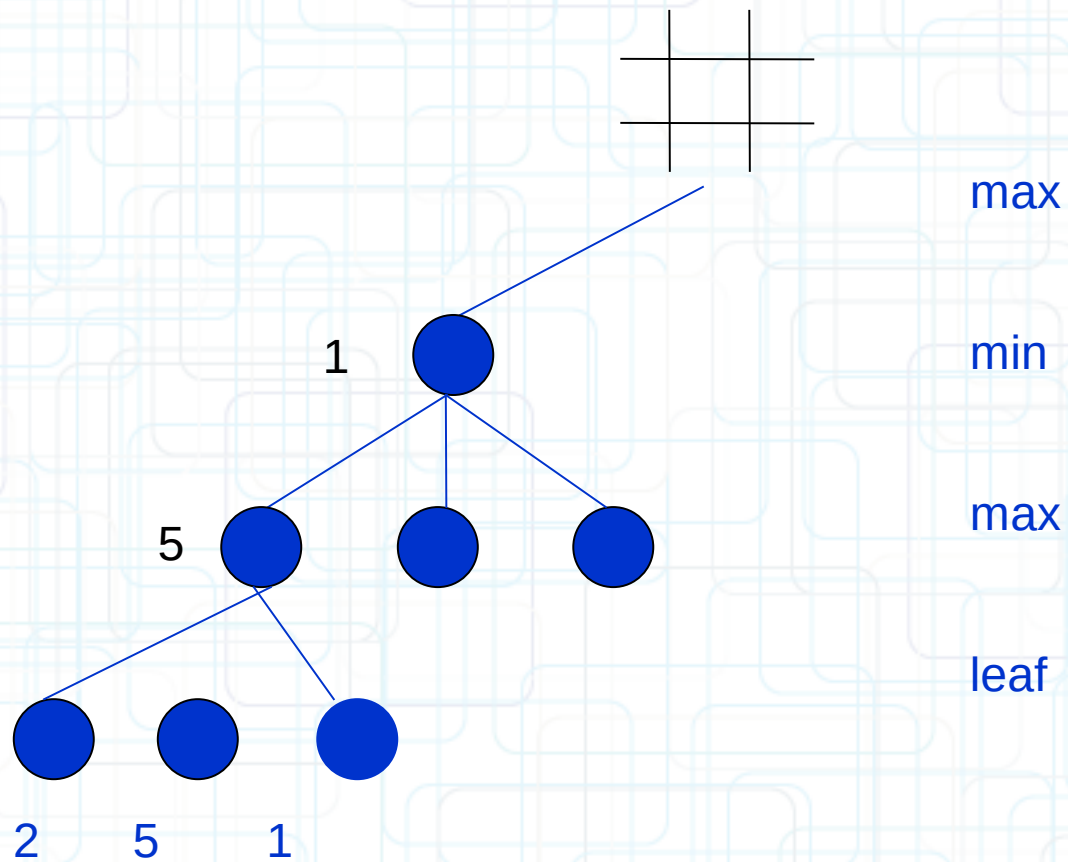
- Матрицаар илэрхийлэгддэг. Нэг тоглогчийг эгнээгээр нөгөө тоглогчийн баганаар дүрсэлдэг.
- Тоглогчийн оноог нүд бүрт дүрслэв.
- 1 ялалт, 0 тэнцүү, -1 хожигдол
- Нийлбэр нь 0 байна

		Player II		
		R	P	S
Player I	R	0/0	-1/1	1/-1
	P	1/-1	0/0	-1/1
	S	-1/1	1/-1	0/0

Хоёр тоглогчтой тоглоом



Minmax-ийг depth first-оор ШИЙДВЭЛ...



Хоёр тоглогчтой, Zero-Sum Тоглоом: Тодорхойлолт

- Хоёр тоглогч A (Max) болон B (Min)
- P байршлын олонлог (тоглоомын төлвүүд)
- *Эхлэх байршил* $s \in P$ (тоглоом эхлэхэд)
- Эцсийн байршилууд $T \subseteq P$ (тоглоом дуусахад)
- Төлвүүдийн хоорондын чиглэлтэй холбоосууд E_A (A -н нүүдэллүүд)
- Төлвүүдийн хоорондын чиглэлтэй холбоосууд E_B (B -н нүүдлүүд)
- Ашиглалтын болон өртөг тооцох функц $U : T \rightarrow \mathbb{R}$ (Тоглогч A -ын хувьд эцсийн төлөв нь хир сайн байгааг тодорхойлно)
- Яагаад бидэнд B -ийн хувьд ашиглалтын буюу өртөг тооцох функц хэрэгтэй вэ?

Minimax algorithm

function MINIMAX-DECISION(*state*) *returns an action*

$v \leftarrow \text{MAX-VALUE}(\textit{state})$

return the *action* in SUCCESSORS(*state*) with value *v*

function MAX-VALUE(*state*) *returns a utility value*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$

return *v*

function MIN-VALUE(*state*) *returns a utility value*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

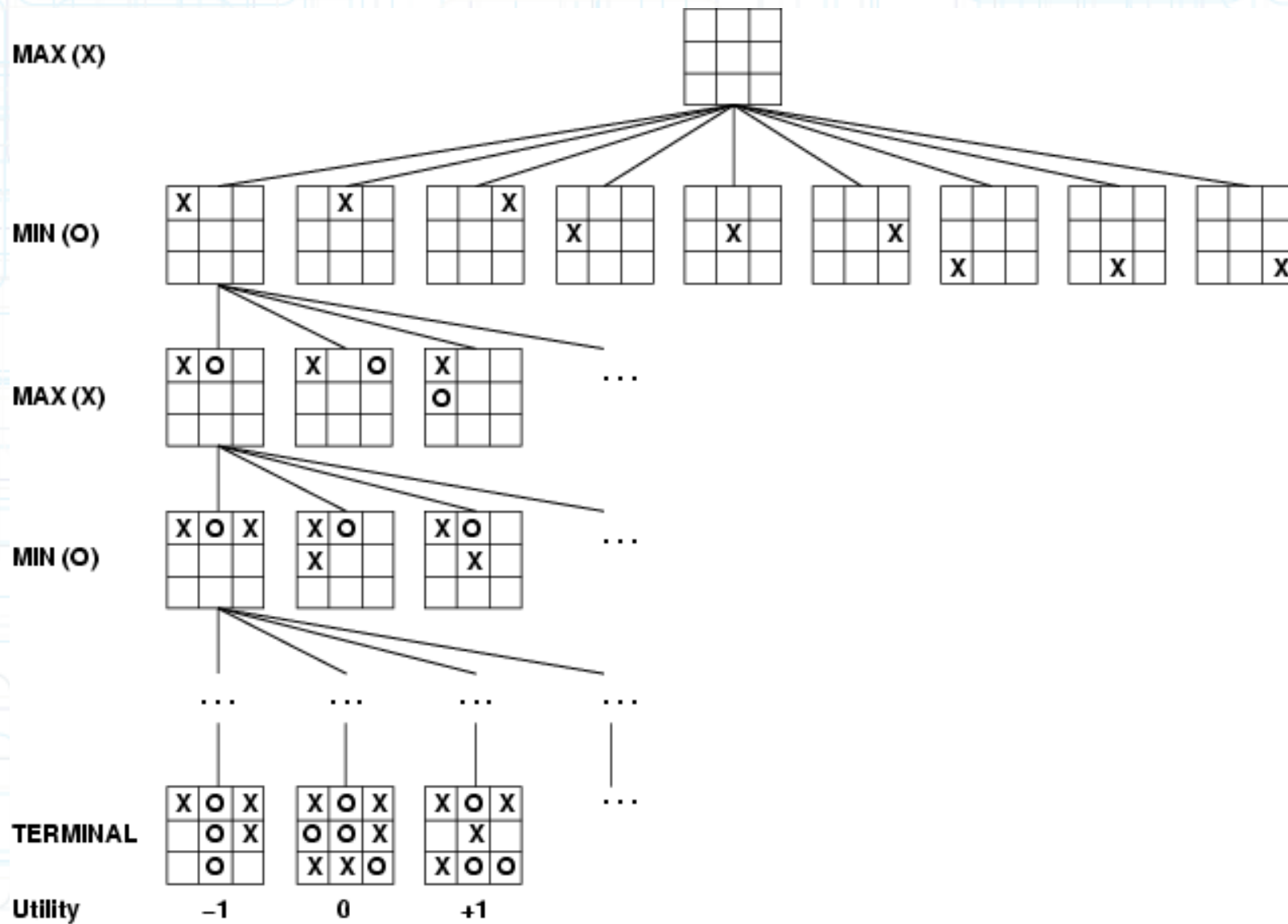
$v \leftarrow \infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$

return *v*

TIC TAC TOE



Minimax-ийн шинж чанар

- Төгс? Тийм (хэрвээ мод нь төгсгөлөг бол)
- Оптималь? Тийм (оптималь өрсөлдөгчийн эсрэг)
- Хугацааны төвөгтэй байдал? $O(b^m)$
- Орон зайн төвөгтэй байдал? $O(bm)$ (depth-first хайлт)

Шатрын хувьд, $b \approx 35$, $m \approx 100$ -аар боломжит тоглоомыг дүрсэлдэг.

→ нарийн жинхэнэ шийдэл нь бүрэн хэрэгждэггүй.

Minimax-ийн шинж чанар

- Төгс? Тийм (хэрвээ мод нь төгсгөлөг бол)
- Оптималь? Тийм (оптималь өрсөлдөгчийн эсрэг)
- Хугацааны төвөгтэй байдал? $O(b^m)$
- Орон зайн төвөгтэй байдал? $O(bm)$ (depth-first хайлт)

Шатрын хувьд, $b \approx 35$, $m \approx 100$ -аар боломжит тоглоомыг дүрсэлдэг.

→ нарийн жинхэнэ шийдэл нь бүрэн хэрэгждэггүй.

Alpha-Beta Процедур

- **Шатар:**
 - Муу байрлалуудыг өргөтгөлгүй алгасдаг хайлт.
 - Хурдтай хайдаг.
 - Шатар хийхдээ энэ алгоритмыг ашиглаж болно.

Alpha-Beta Процедур

- Муу байрлал гэж юу вэ?
- Шатарт муу нүүдэл гэж юу вэ?



Alpha-Beta Процедур

- Муу байрлал гэж юу вэ?
- Шатарт муу нүүдэл гэж юу вэ?



Alpha-Beta Процедур

- Alpha-beta процедур нь depth-first minimax хайлтыг хурдасгадаг.
- Alpha: хамгийн их утгатай max зангилааг тодорхойлох утгын доод хязгаар
$$v > \alpha$$
- Beta: хамгийн бага утгатай min зангилааг тодорхойлох утгын дээд хязгаар
$$v < \beta$$
- Хэрэв beta утга нь alpha (MAX эцэг зангилаанууд) -аас бага буюу тэнцүү байхад хамгийн бага MIN зангилааг alpha-pruned болгоно.
- Хэрэв alpha утга нь beta (MIN эцэг зангилаанууд) -аас их буюу тэнцүү байхад хамгийн их MAX зангилааг beta-pruned болгоно.

Alpha-Beta –ийн процедур

Альфа-бета процедур нь хамгийн бага $\min\max$ хайлтыг хурдасгаж чаддаг.

Alpha: хамгийн их утгатай зангилааг зааж өгч болох утгын доод хязгаар

$$v \geq \alpha$$

Beta: хамгийн бага утгатай зангилааг зааж өгч болох утгын дээд хязгаар

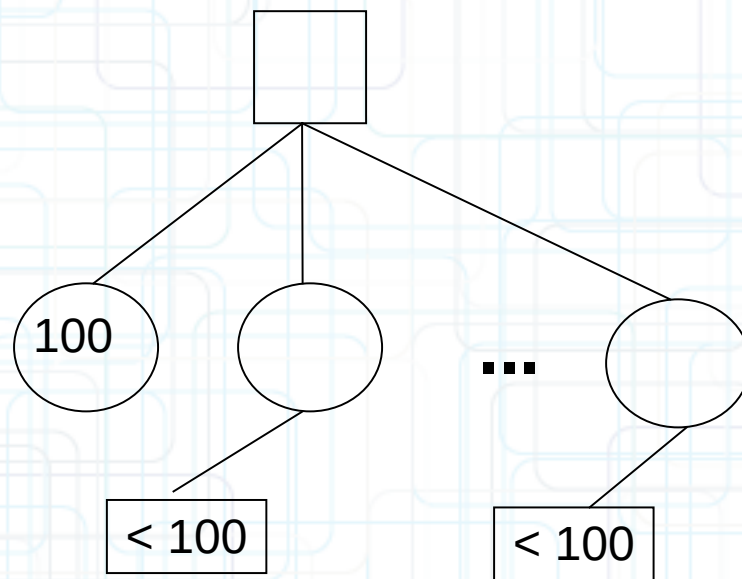
$$v \leq \beta$$

α - β тайралт

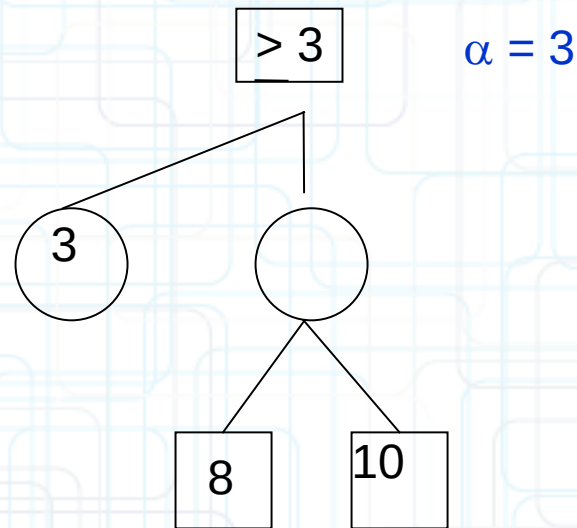
Ажиллах зарчим

- α нь **Макс** хүрэх зам дагуу олдсон хамгийн сайн (хамгийн өндөр) юм
- β нь **Min**-ийн зам дагуу өнөөг хүртэл олдсон хамгийн сайн (хамгийн бага) хэмжээ юм

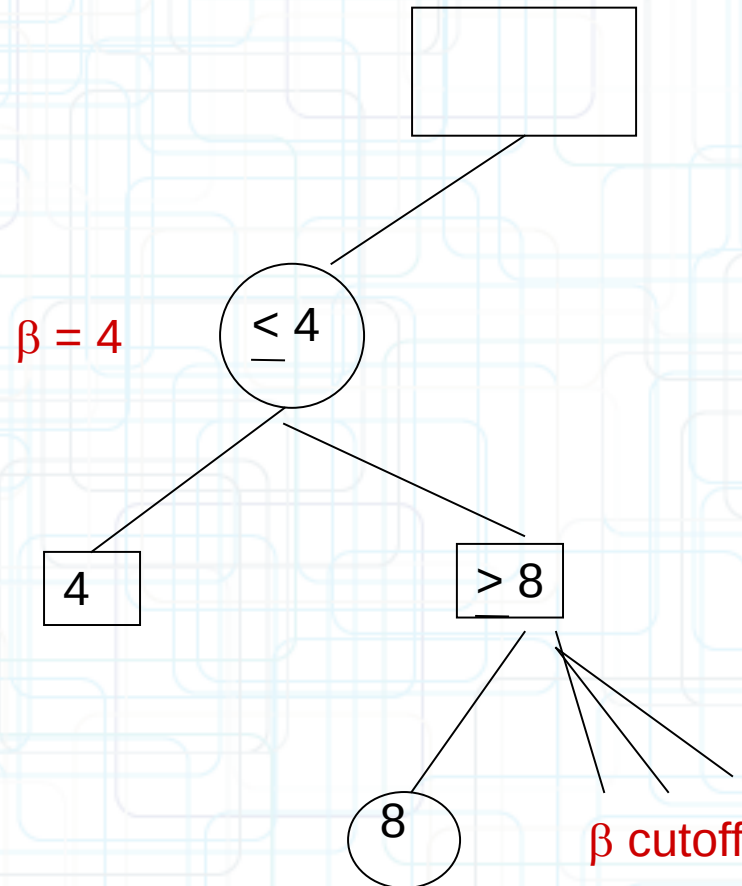
Хэзээ alpha-г таслах вэ?



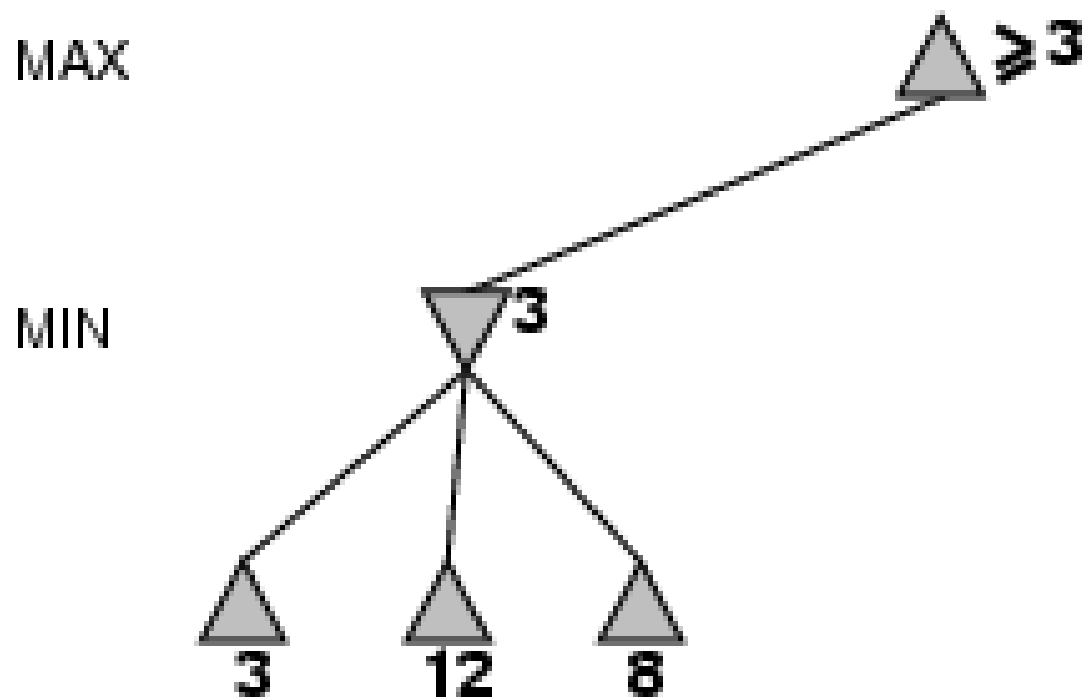
Alpha Cutoff



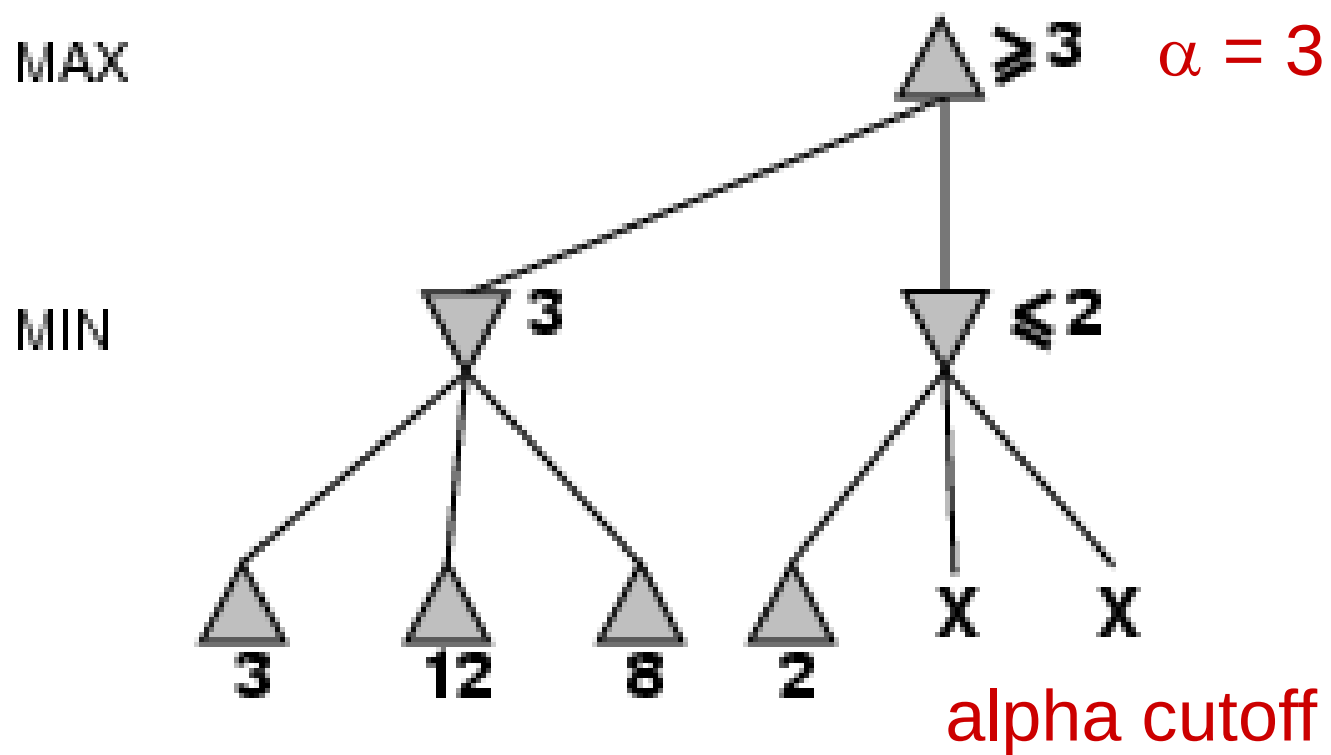
Alpha Cutoff



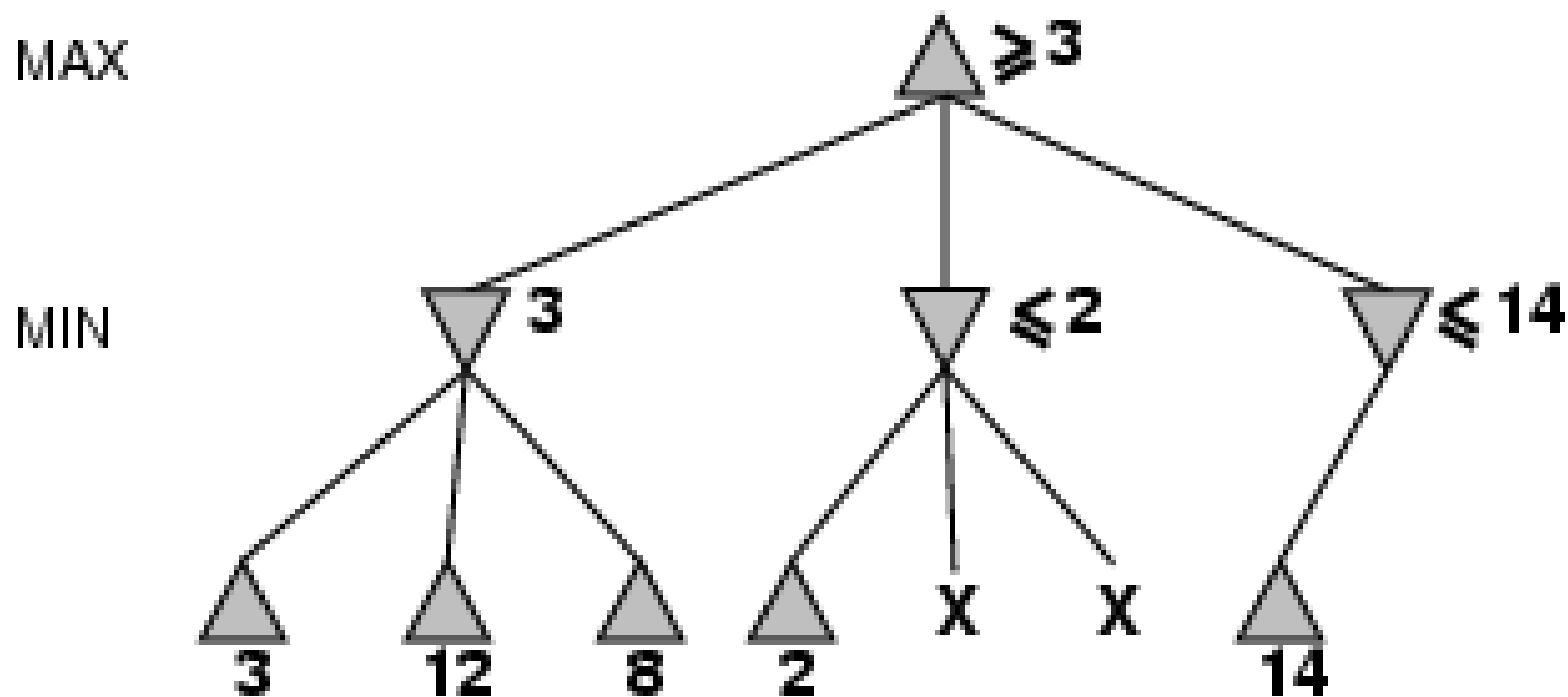
α - β pruning жишээ



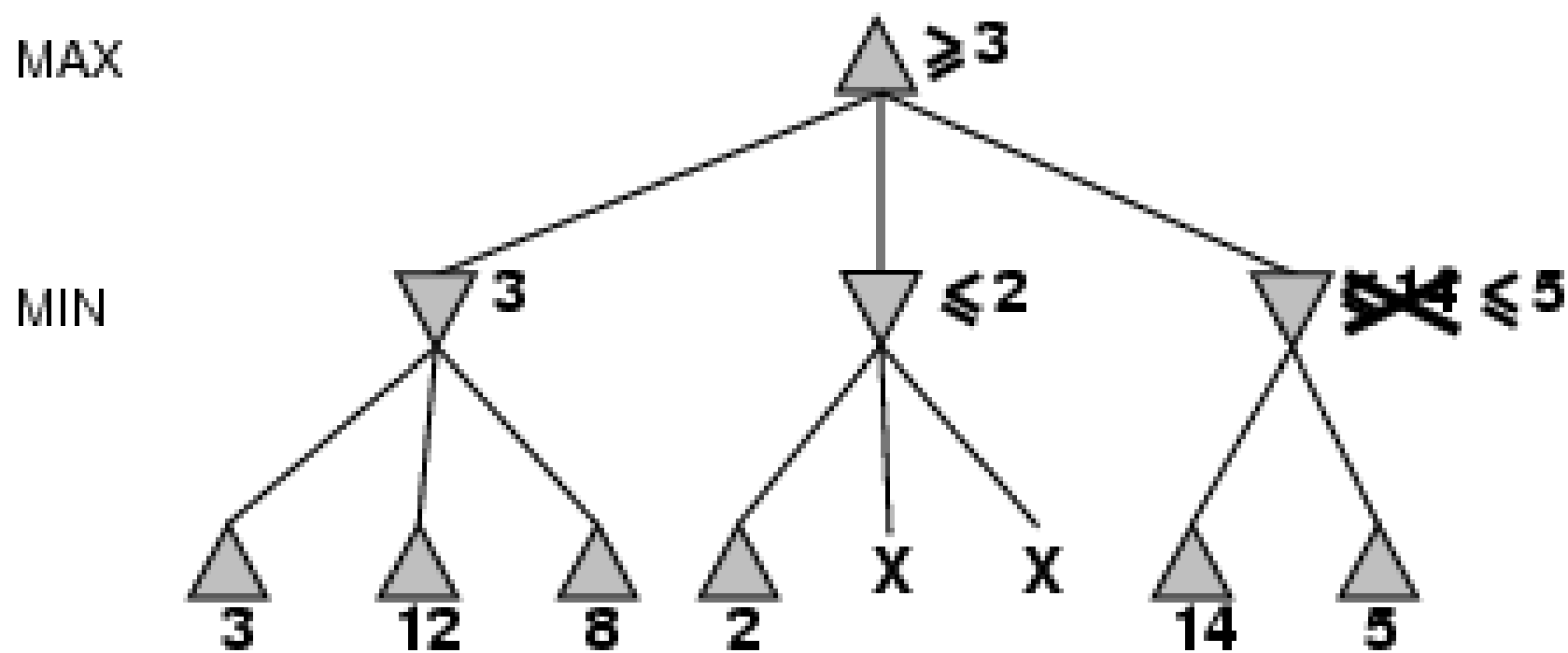
α - β pruning жишээ



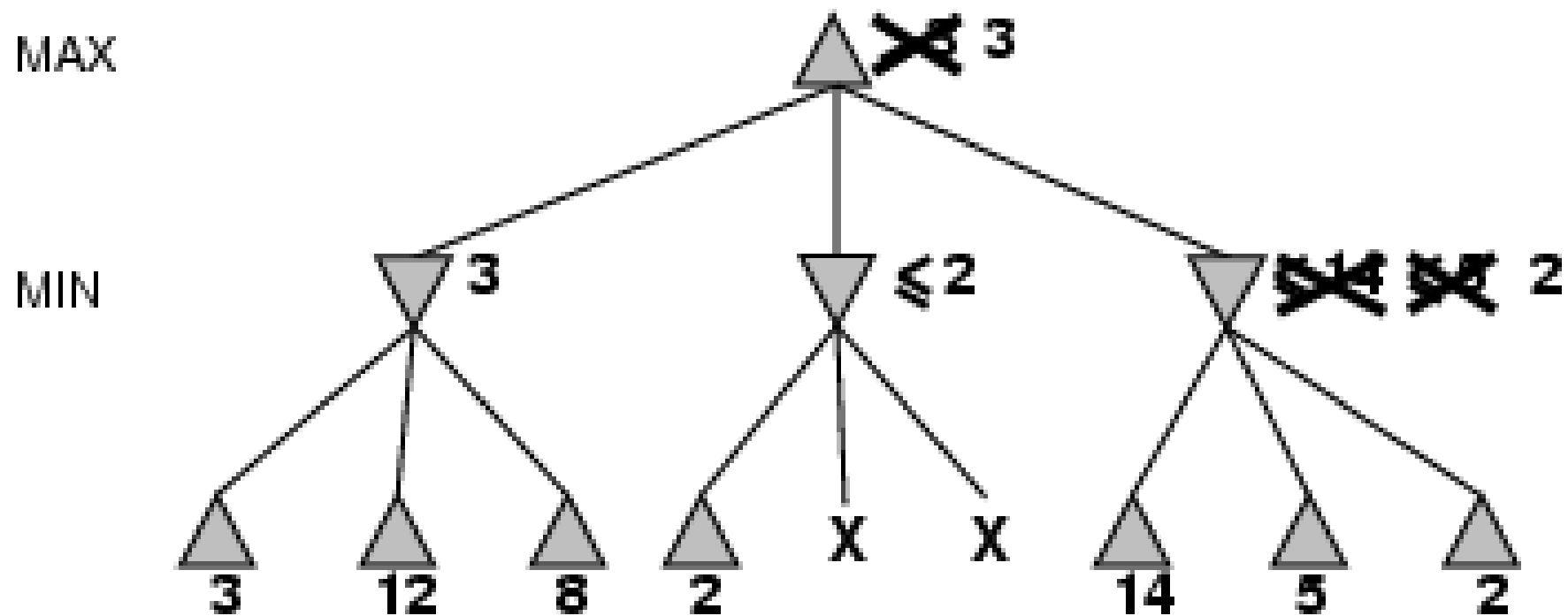
α - β pruning жишээ



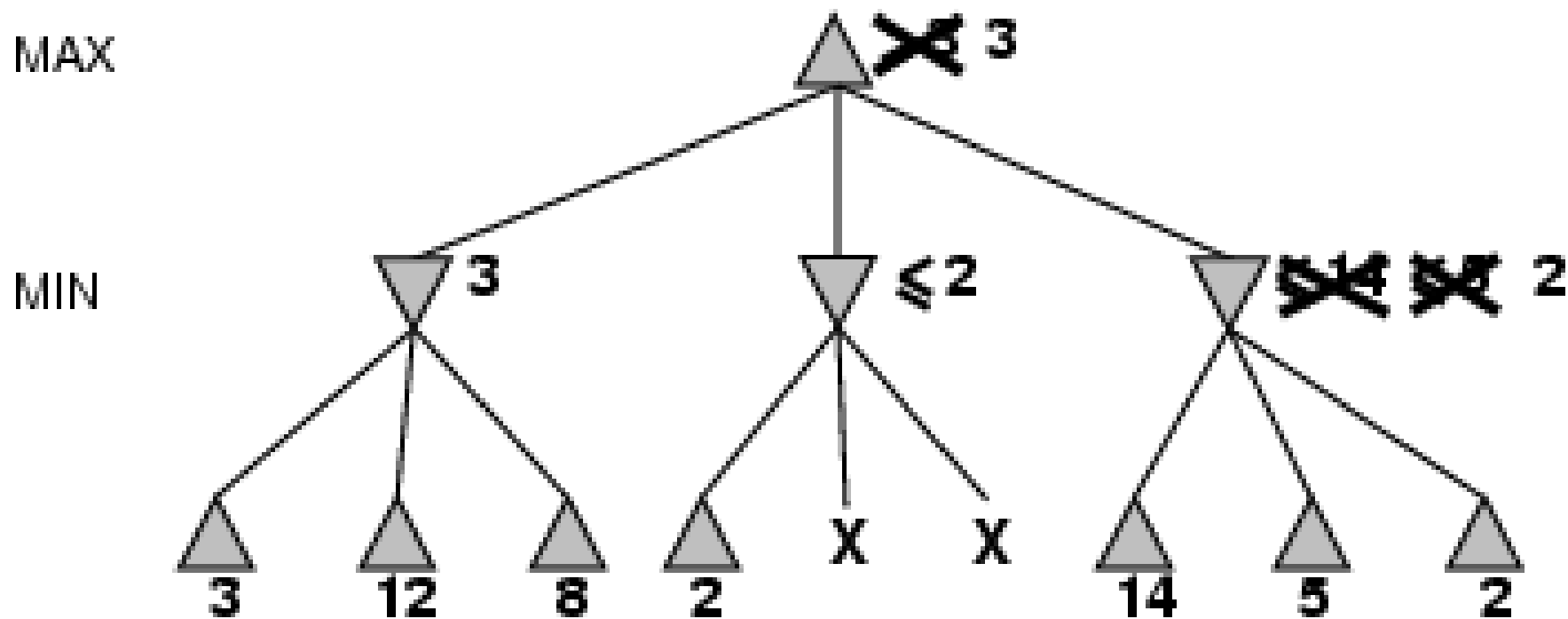
α - β pruning жишээ



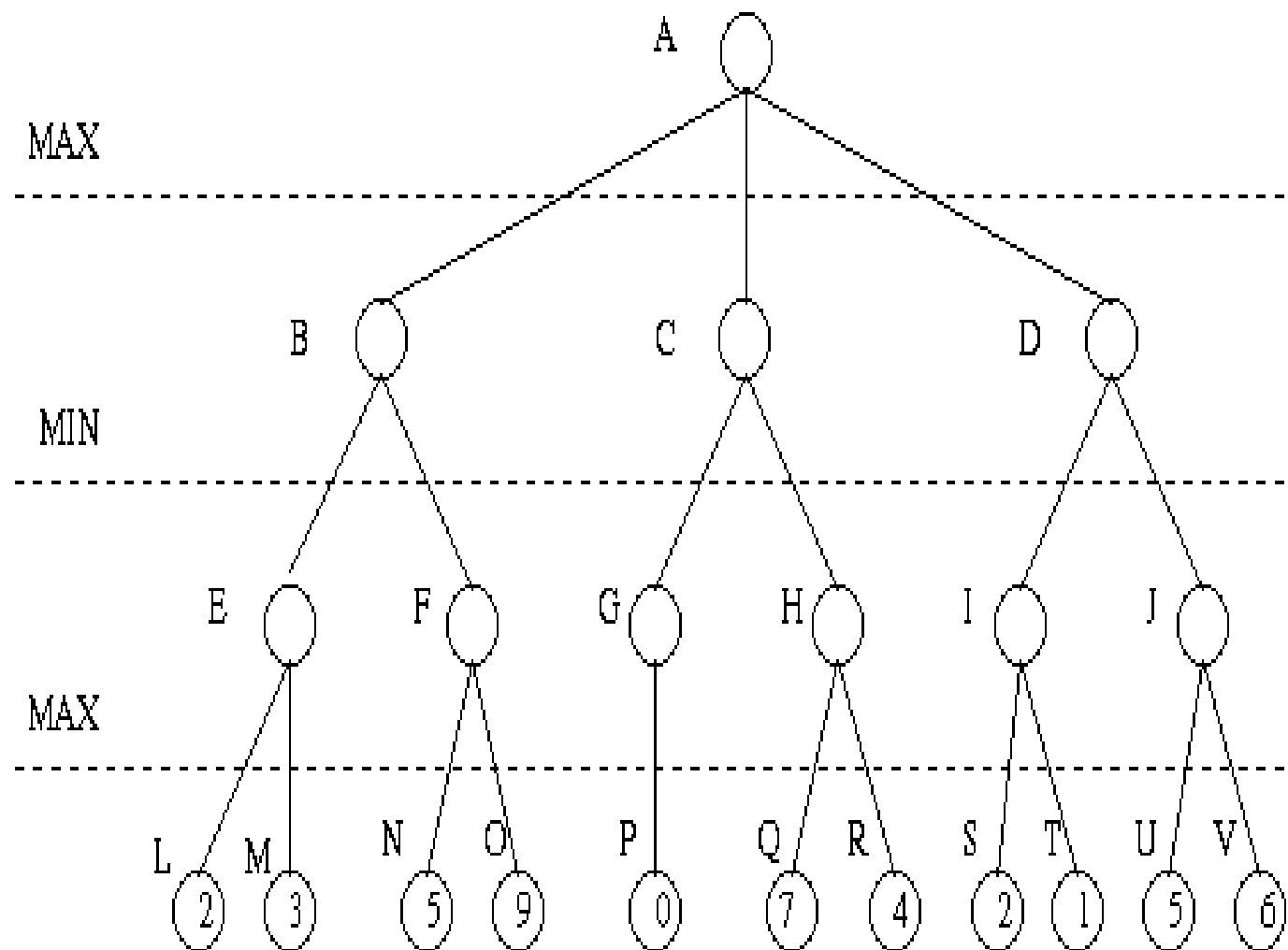
α - β pruning жишээ



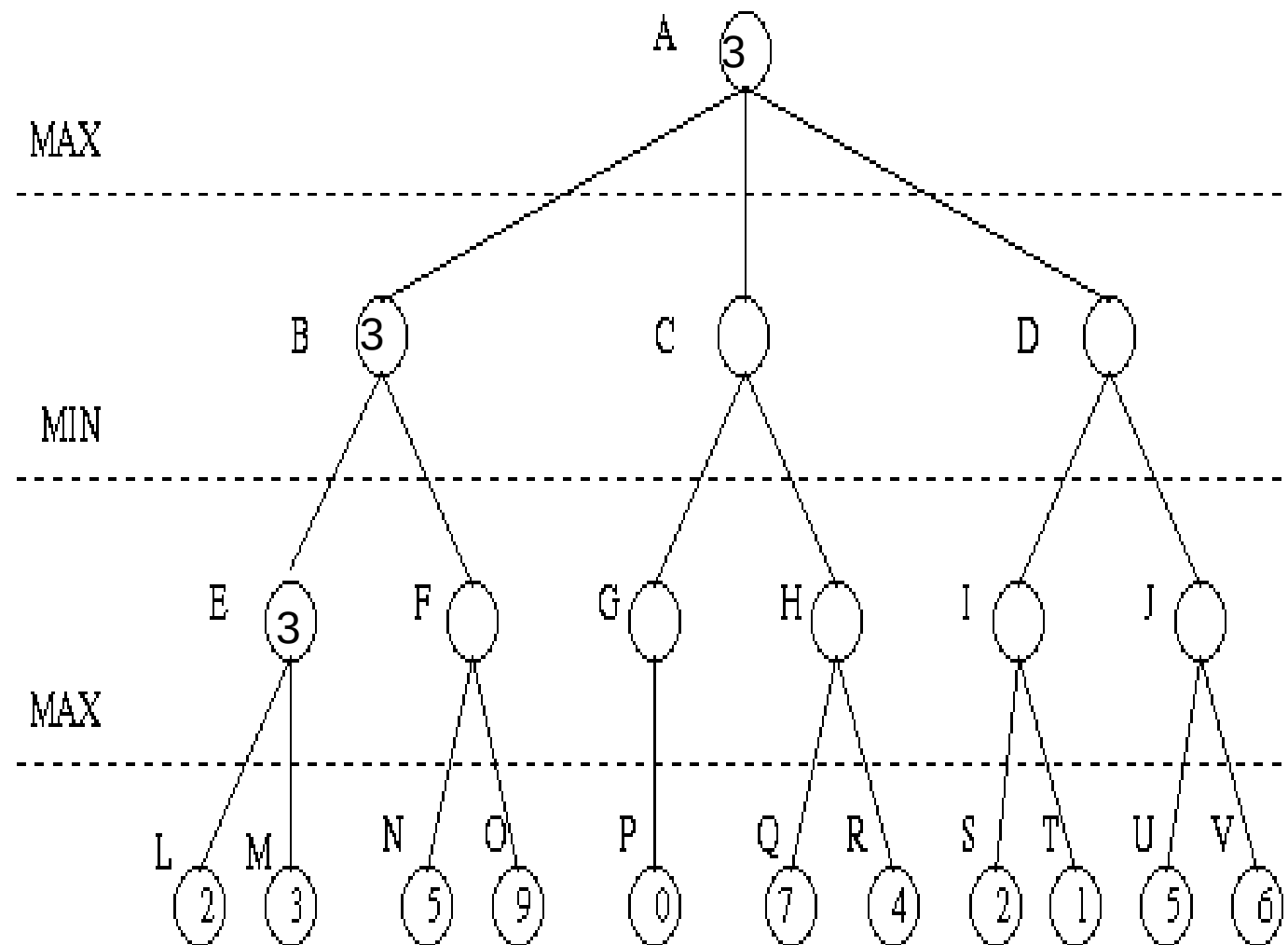
α - β pruning жишээ



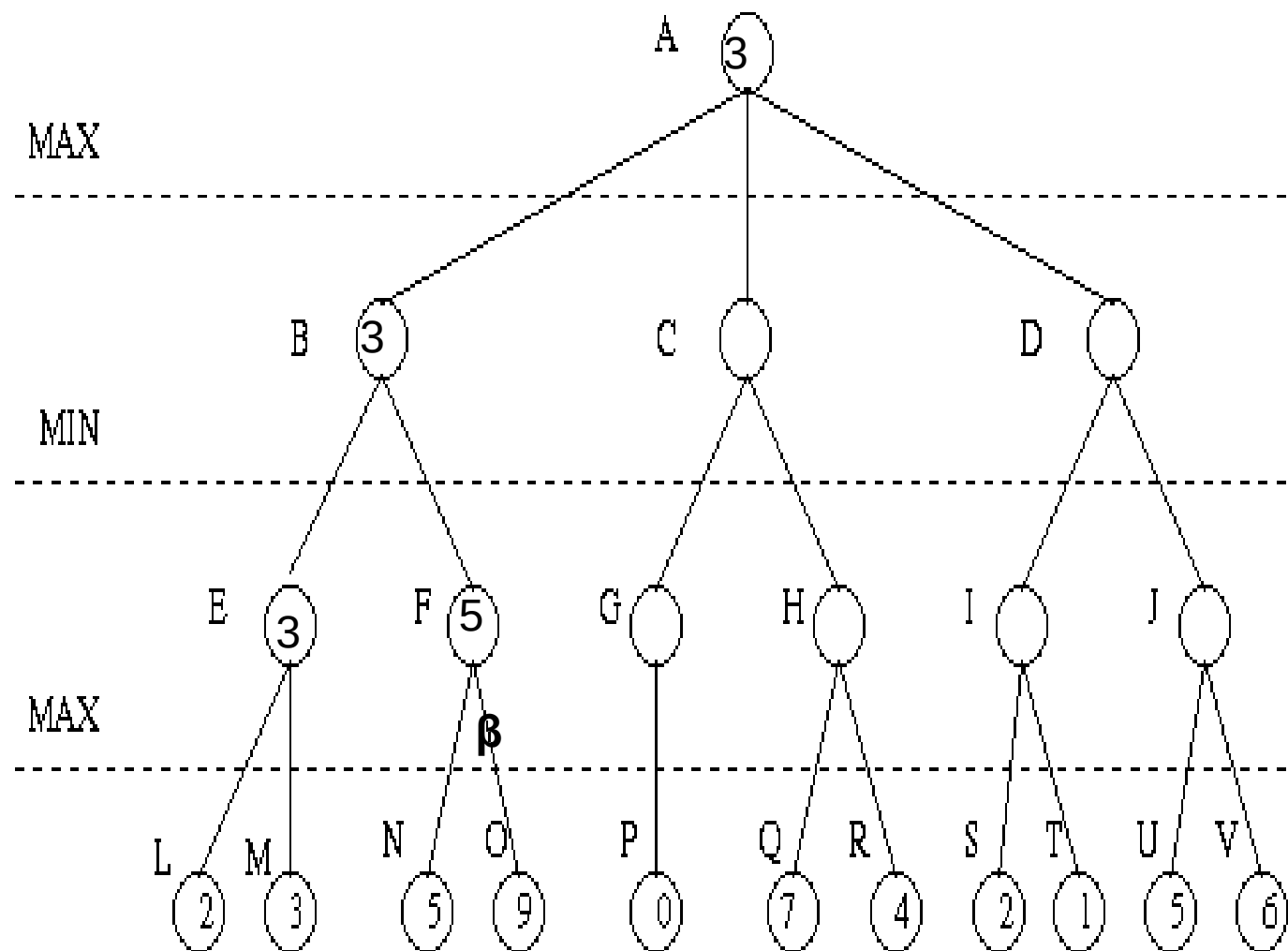
Alpha-Beta Процедур



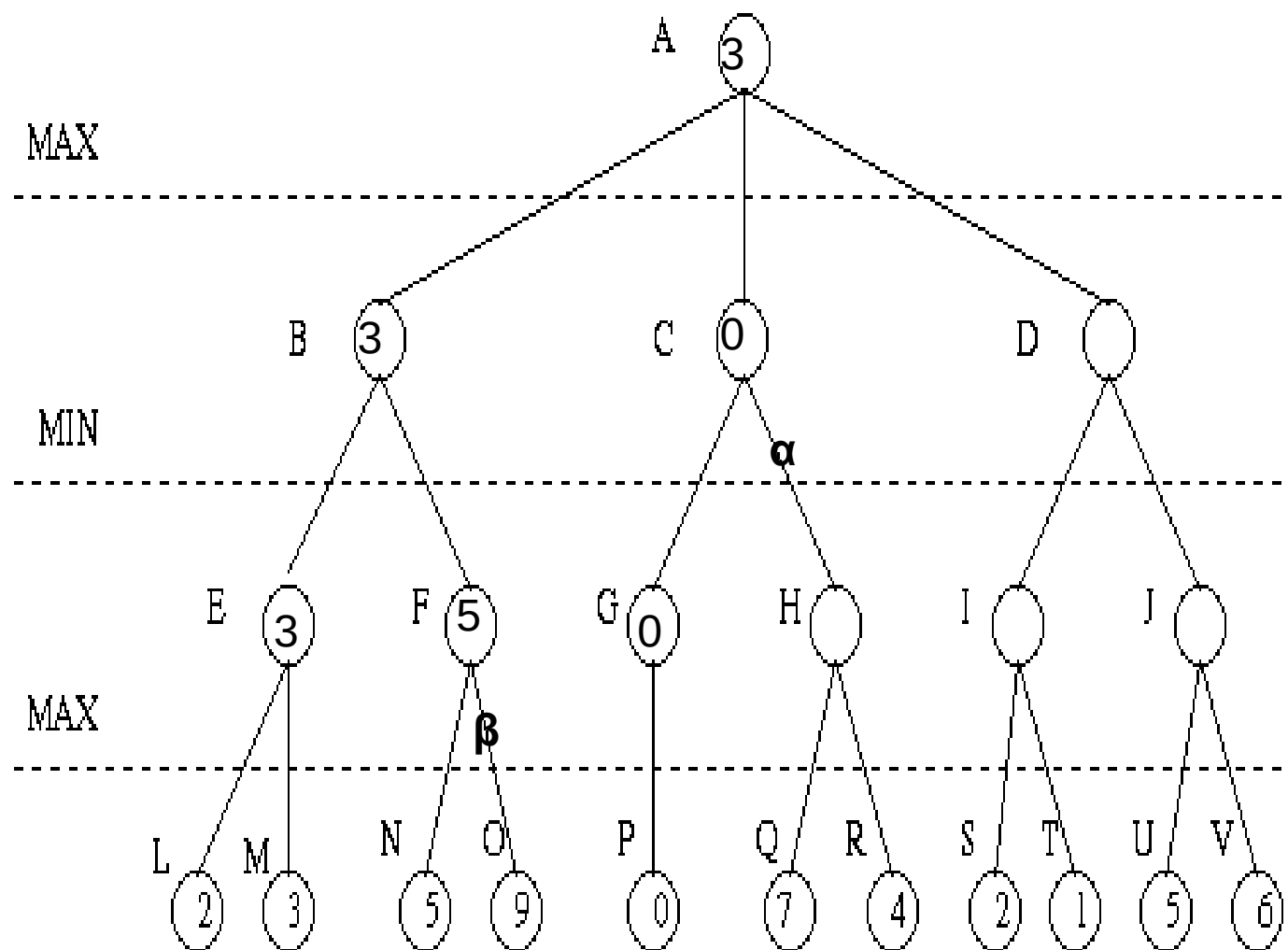
Alpha-Beta Процедур



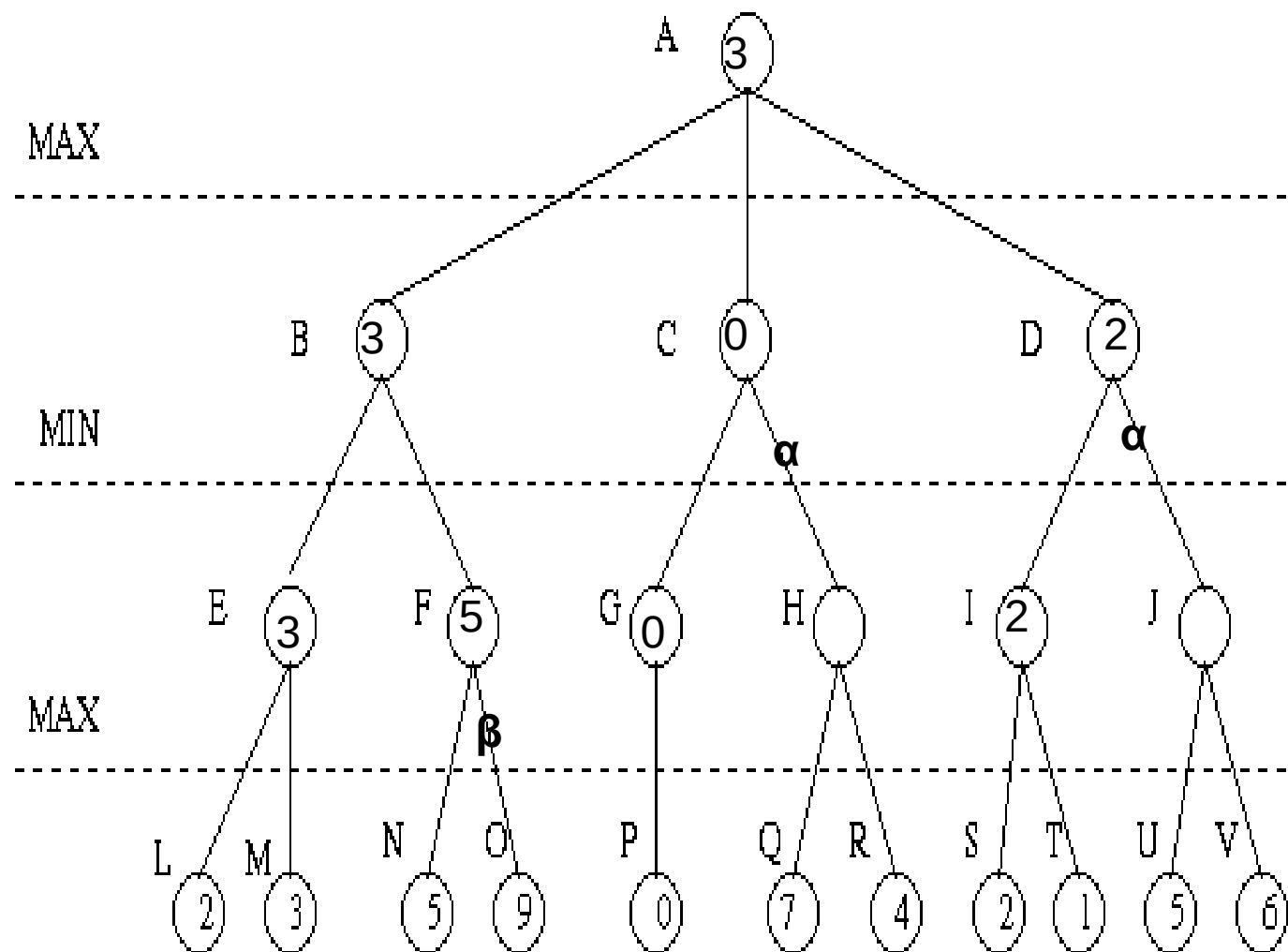
Alpha-Beta Процедуры



Alpha-Beta Процедур



Alpha-Beta Процедур



α - β pruning алгоритм

function ALPHA-BETA-SEARCH(*state*) *returns an action*

inputs: *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the *action* in SUCCESSORS(*state*) with value v

function MAX-VALUE(*state*, α , β) *returns a utility value*

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for a, s in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$

if $v \geq \beta$ **then return** v

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return v

α - β pruning алгоритм

function MIN-VALUE(*state*, α , β) *returns a utility value*

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow +\infty$

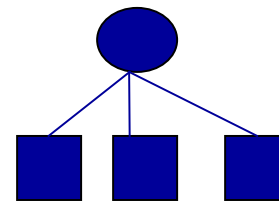
for a, s in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$

if $v \leq \alpha$ **then return** v cutoff

$\beta \leftarrow \text{MIN}(\beta, v)$

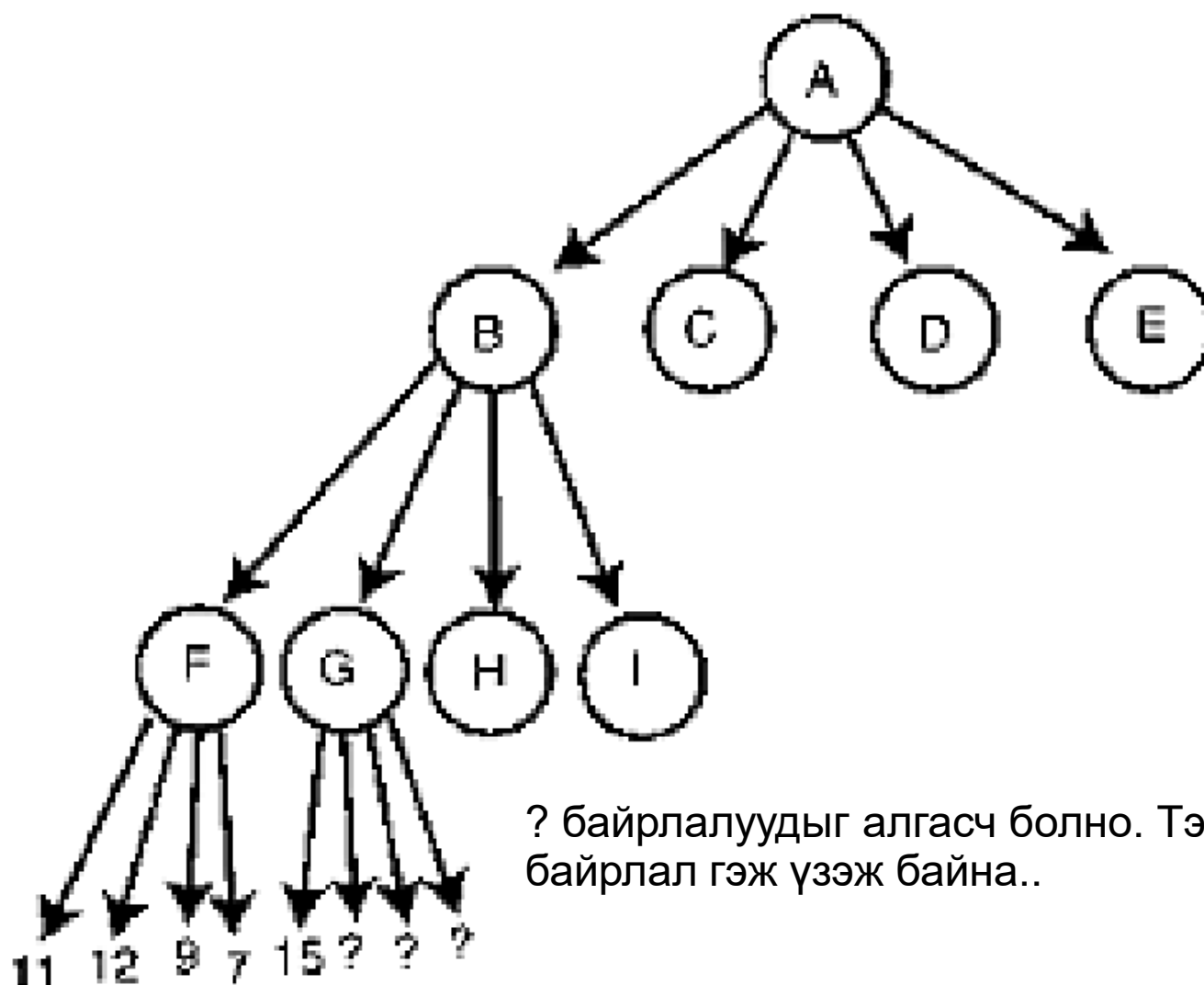
return v



α - β pruning шинж чанар

- Pruning эцсийн үр дүнд нөлөөлөхгүй бөгөөд бүтэн minimax-тай яг ижил эцсийн үр дүнтэй байдаг.
- Сайн нүүдэлийн эрэмбэлэлт pruning-ийн үр ашигийг нэмэгдүүлдэг.
- Төгс эрэмбэлэлт-ийн хугацааны төвөгтэй байдал нь $O(b^{m/2})$ байдаг. Энэ нь гүнээр хайлтыг хоёр дахин нэмэгдүүлдэг.

α - β pruning хайлт



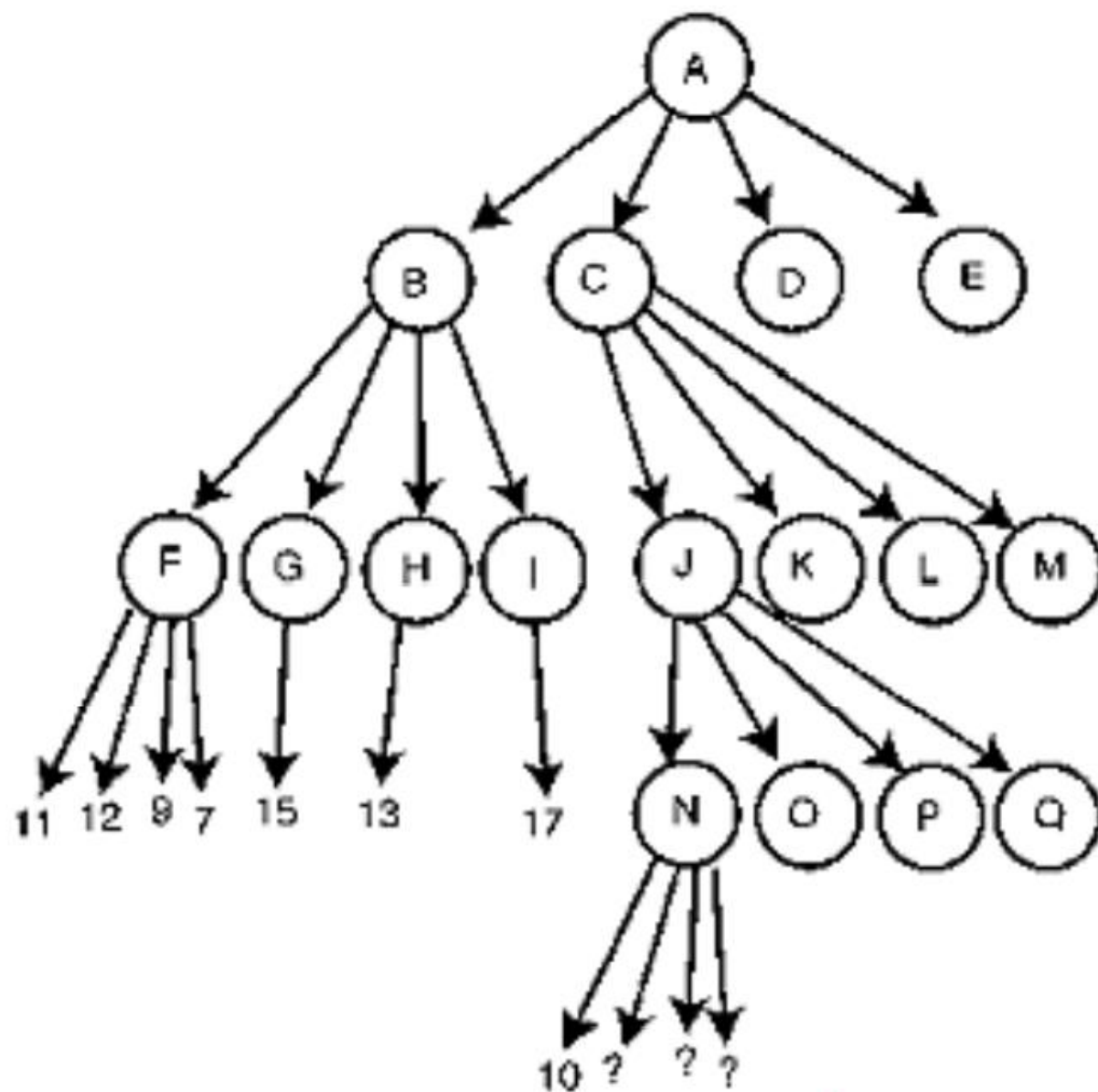
α - β pruning шинж чанар

- Хайлтаар F зангилаанд хүрээд түүний хүүхдүүдийн үнэлгээ нь 11, 12, 7, 9 болохыг тогтоожээ. Хайлтын энэ түвшинд эхний тоглогч нүүх байгаа бөгөөд бид түүнд хамгийн сайн нүүдэл болох 12 – г сонгоно. Иймээс F ийн үнэлгээ нь 12 болж байгаа юм.
- Одоо бид G зангилааг өргөтгөж эхлэх бөгөөд түүний эхний хүүхдийн үнэлгээ нь 15 байгааг зургаас харж болно. Энэ нь юу гэсэн үг вэ гэвэл G –ийн үнэлгээ нь ямарч байсан 15 –с илүү байна (хамгийн багадаа 15) гэсэн үг юм.

α - β pruning хайлт

- Хэрэв бид хоёрдугаар тоглогчийн нүдээр харвал G нүүдлийг хийхгүй бөгөөд учир нь түүнээс сайн F нүүдэл байна. (Хоёрдугаар тоглогч эхний тоглогчид бага оноо өгөхийн төлөө тоглох тул) Иймээс G нь үндсэн хувилбар биш болохыг бид мэдэж авах юм.
- Ингээд бид G-н үлдсэн хүү зангилаануудыг өргөтгөлгүйгээр хасан цаашаа H-г өргөтгөхөөр явж болно. Ингэснээр бид олон зангилааг өргөтгөлгүйгээр их цаг хэмнэн хайлтыг үр дүнтэйгээр явуулж чадах бөгөөд энэ нь альфа бетта хайлтын үндсэн санаа юм.

α - β pruning хайлт



“Альфа Бетта хасах” хайлтын алгоритм

```
double alphabeta(int depth, double alpha, double beta)
```

```
{
```

```
    if (depth <= 0 || тоглоом дууссан) return evaluation();
```

энэ байрлалд боломжтой байгаа нүүдлүүдийг үүсгэн эрэмбэлнэ.

```
    for (m нүүдэл бүрийн хувьд) {
```

```
        m нүүдлийг гүйцэтгэ.
```

```
        double val = -alphabeta(depth - 1, -beta, -alpha);
```

```
        m нүүдлийг буцах;
```

```
        if (val >= beta) return val;
```

```
        if (val > alpha) alpha = val;
```

```
    }
```

```
    return alpha;
```

```
}
```

α - β pruning алгоритм

function ALPHA-BETA-SEARCH(*state*) *returns an action*

inputs: *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the *action* in SUCCESSORS(*state*) with value v

function MAX-VALUE(*state*, α , β) *returns a utility value*

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for a, s in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$

if $v \geq \beta$ **then return** v

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return v

α - β pruning алгоритм

function MIN-VALUE(*state*, α , β) *returns a utility value*

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow +\infty$

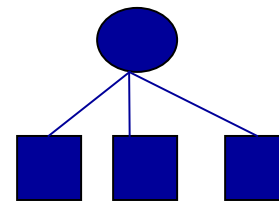
for a, s in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$

if $v \leq \alpha$ **then return** v cutoff

$\beta \leftarrow \text{MIN}(\beta, v)$

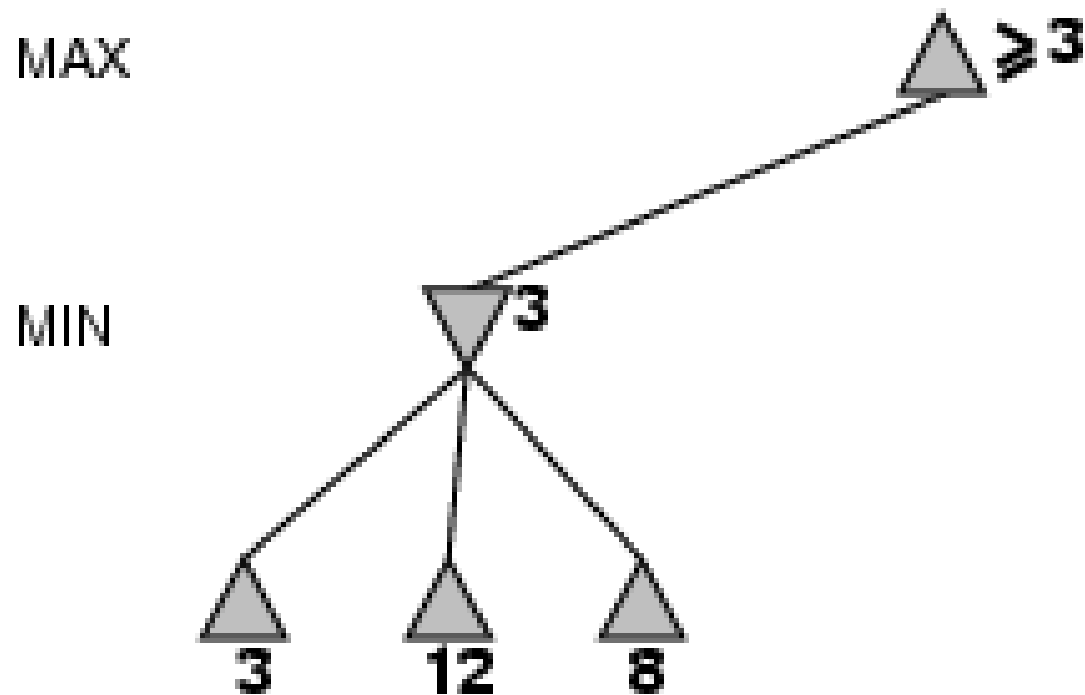
return v



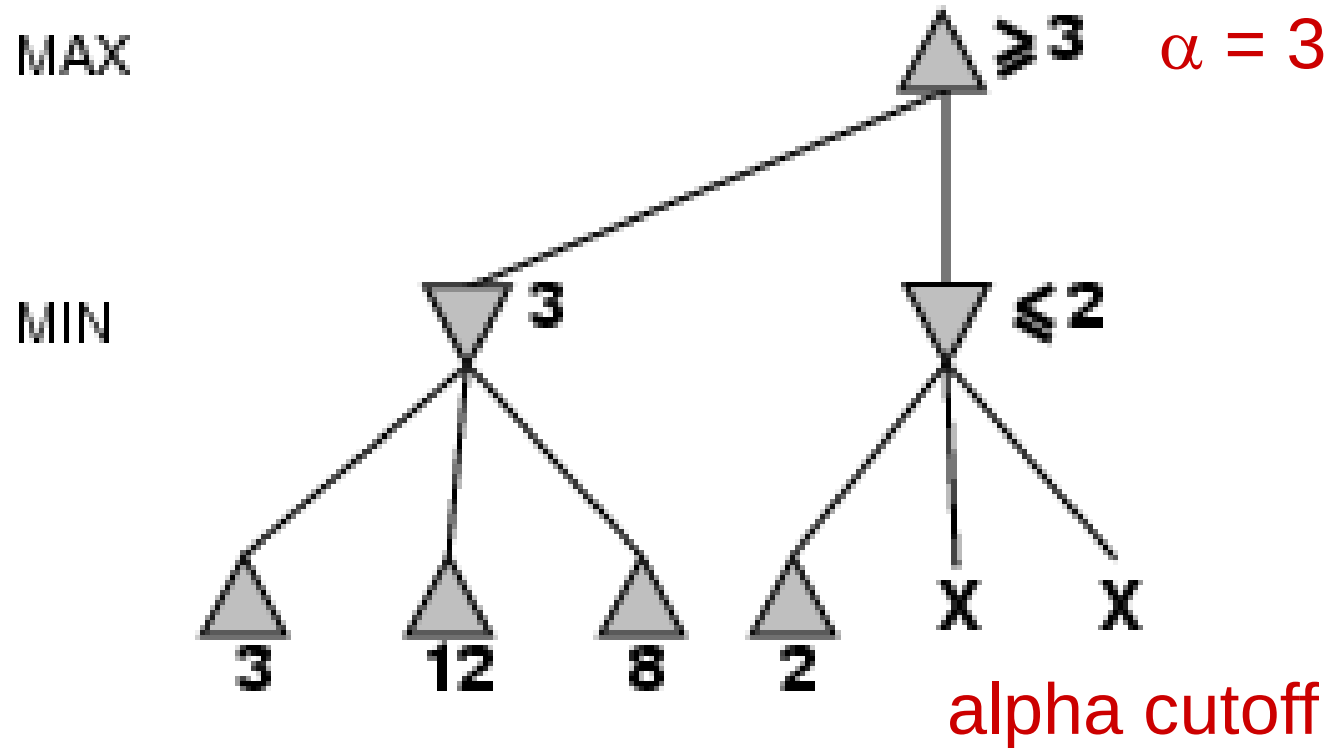
α - β pruning шинж чанар

- Pruning эцсийн үр дүнд нөлөөлөхгүй бөгөөд бүтэн minimax-тай яг ижил эцсийн үр дүнтэй байдаг.
- Сайн нүүдэлийн эрэмбэлэлт pruning-ийн үр ашигийг нэмэгдүүлдэг.
- Төгс эрэмбэлэлт-ийн хугацааны төвөгтэй байдал нь $O(b^{m/2})$ байдаг. Энэ нь гүнээр хайлтыг хоёр дахин нэмэгдүүлдэг.

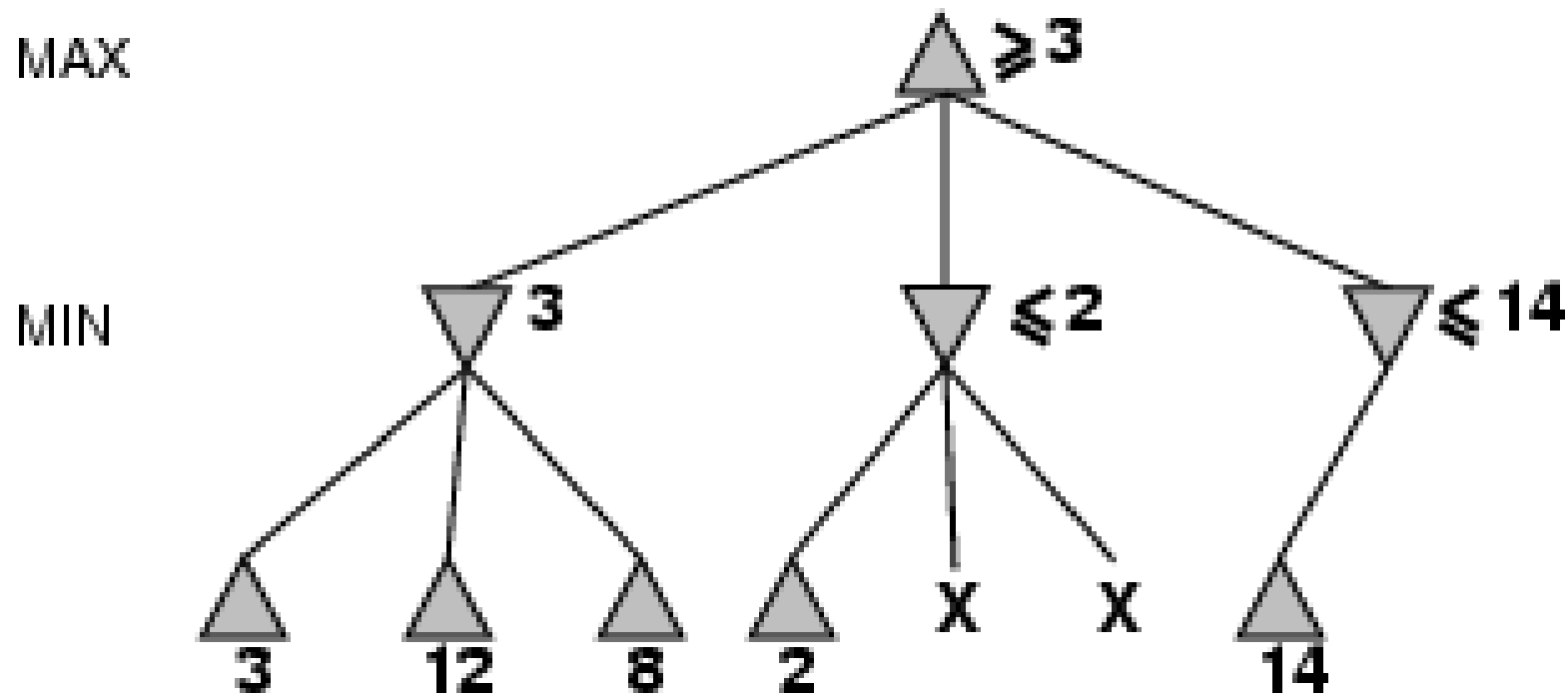
α - β тайралт хийх жишээ



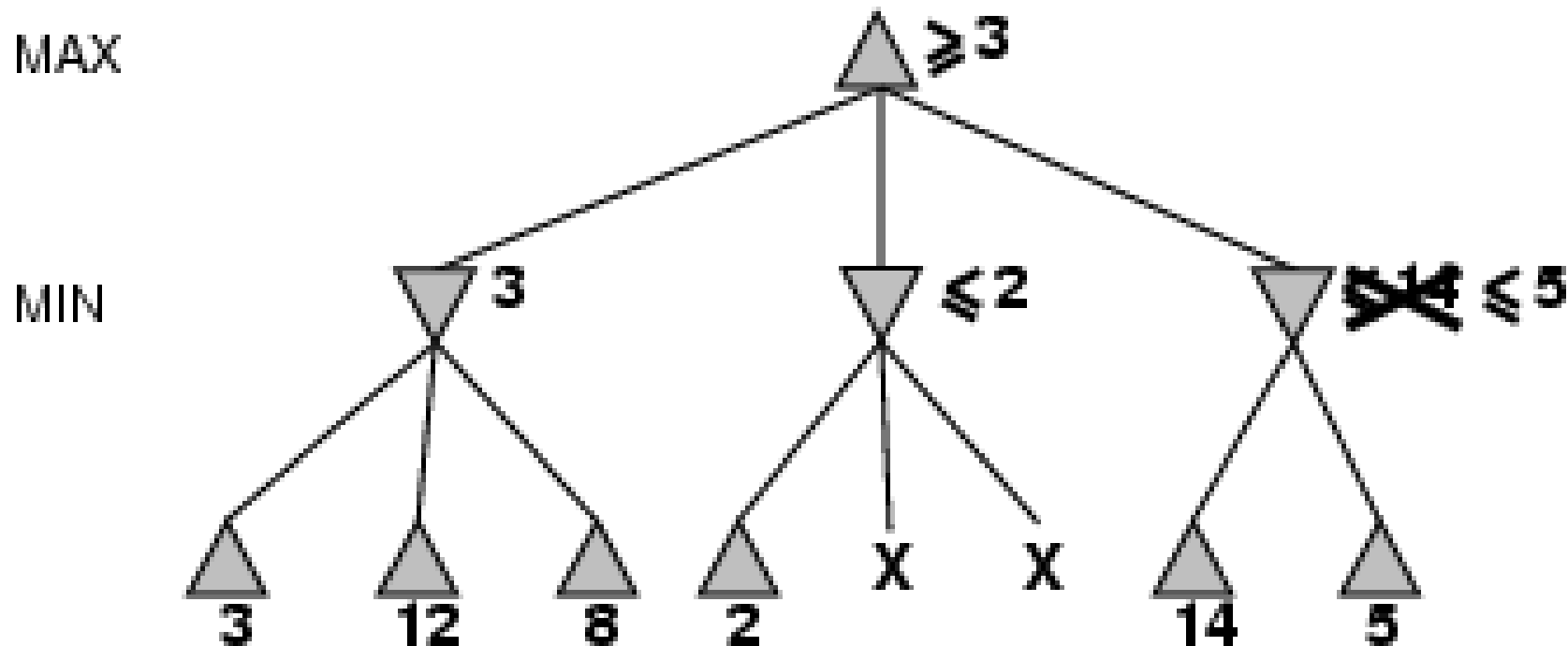
α - β тайралт хийх жишээ



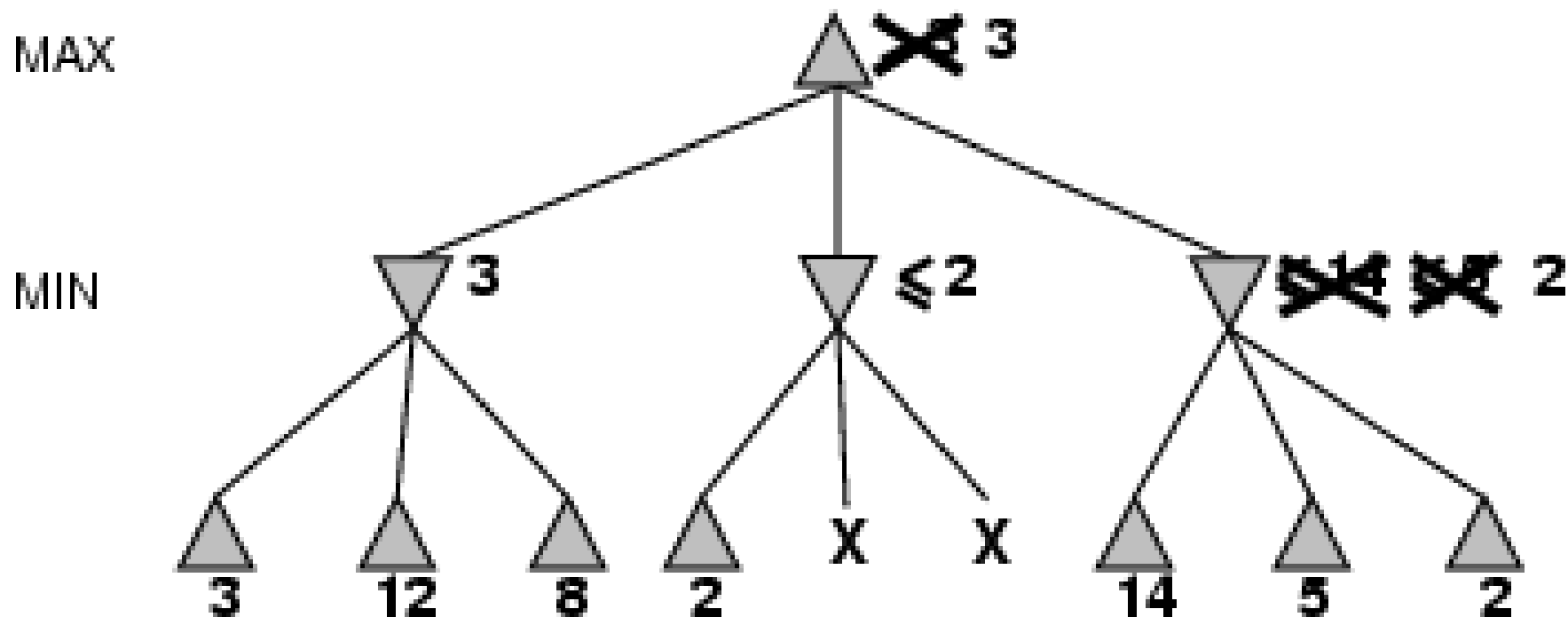
α - β тайралт хийх жишээ



α - β тайралт хийх жишээ



α - β тайралт хийх жишээ



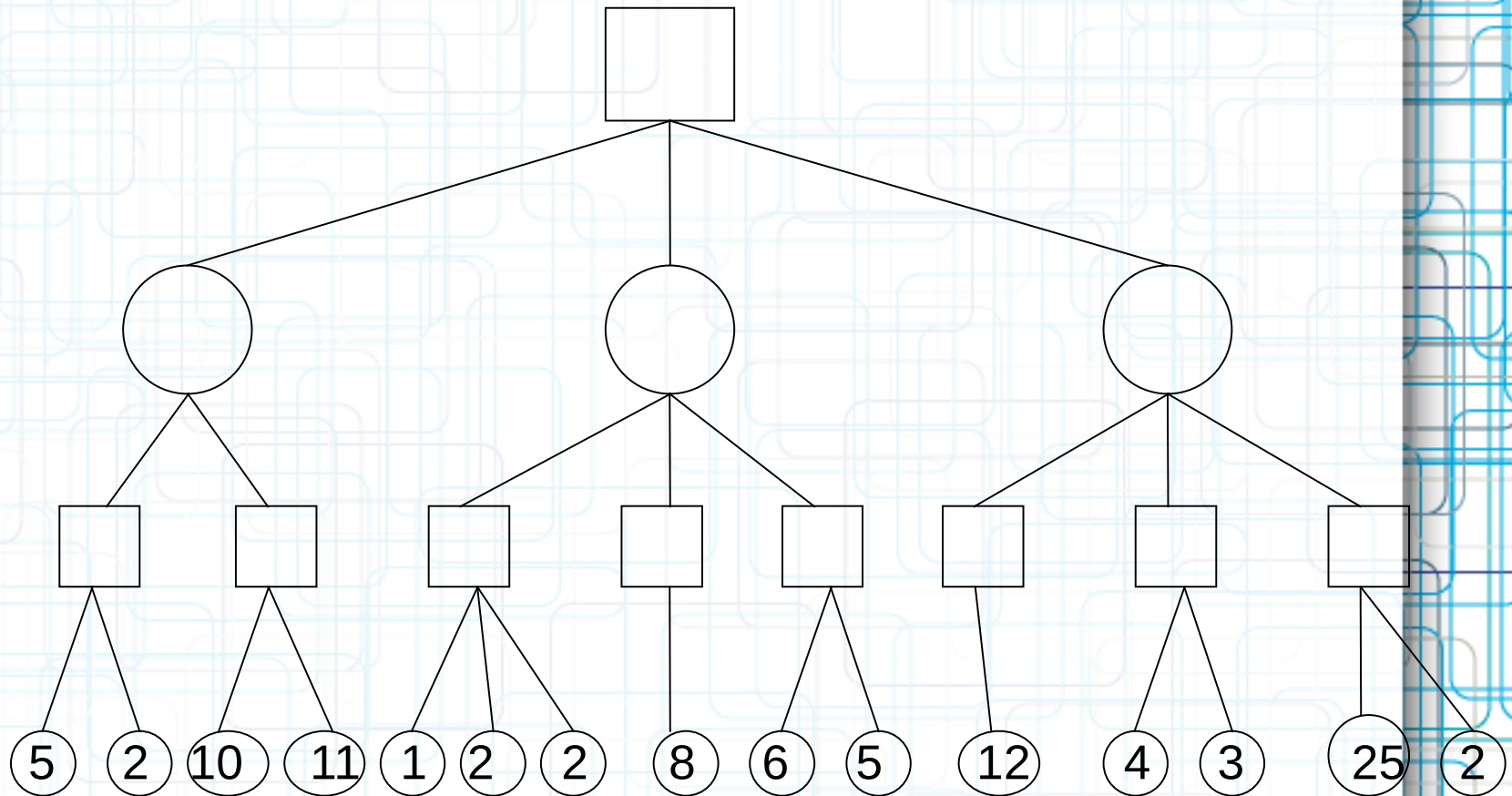
α - β тайралт

max

min

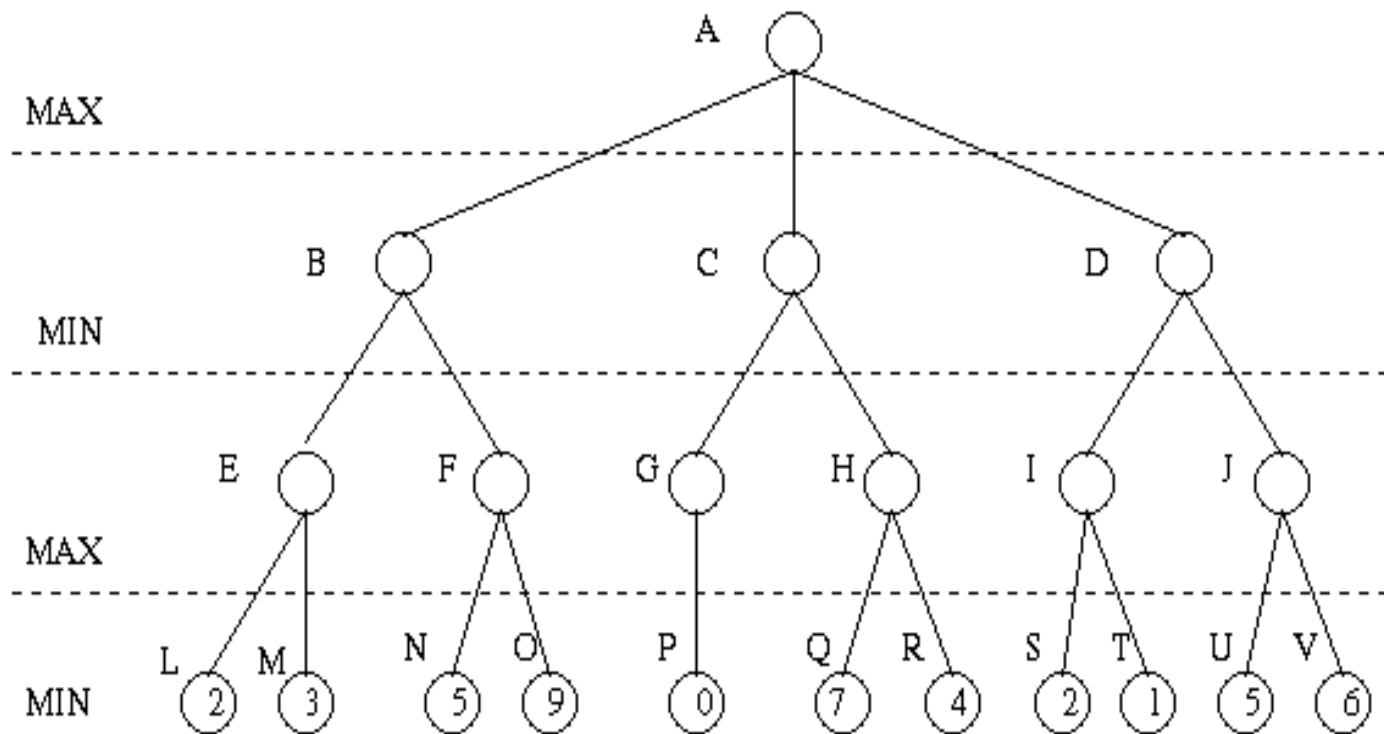
max

eval



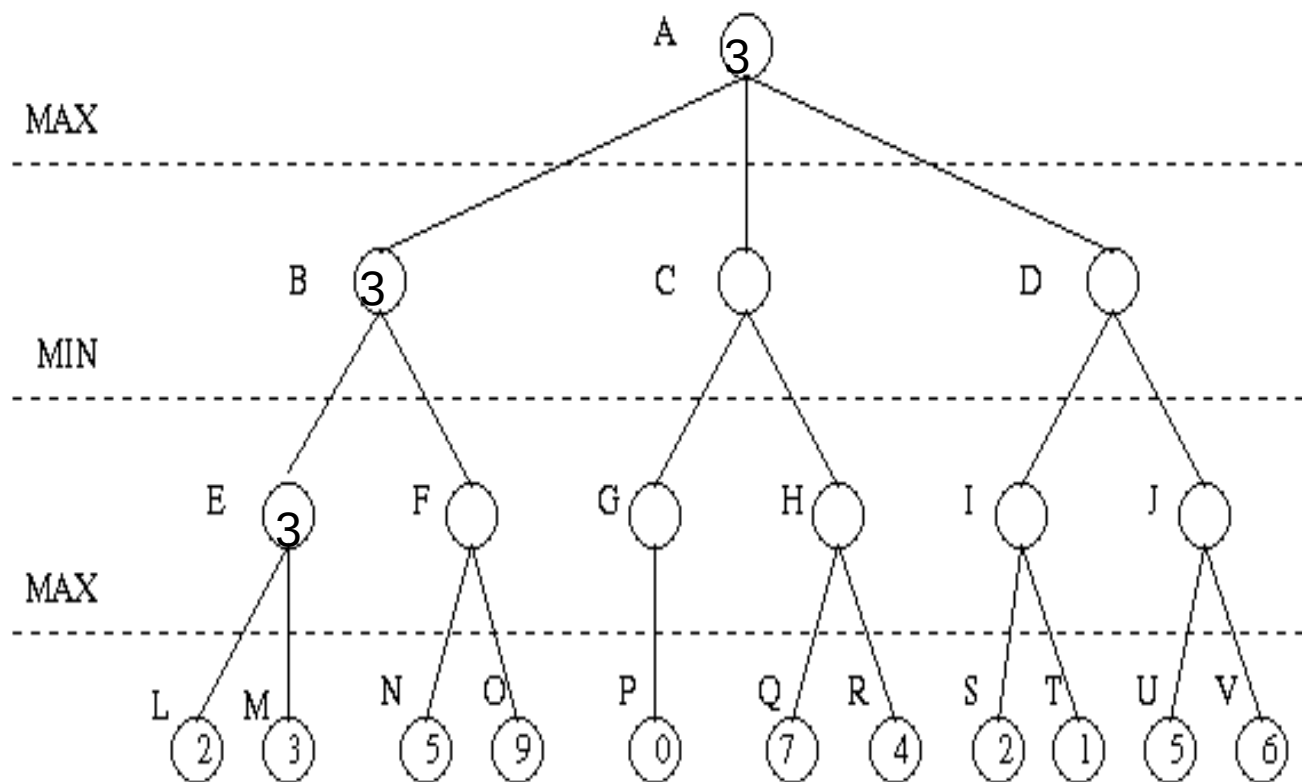
α-β тайралтын жишээ

1. Хэрэв beta утга нь MAX өвөг эцгийн alpha утгаас $\leq$ үед MIN зангилаанаас бага зангилааг хайж alpha тайралт хийнэ.
2. Хэрэв alpha утга нь MIN өвөг эцгийн alpha утгаас $\geq$ үед MIN зангилаанаас бага зангилааг хайж beta тайралт хийнэ.



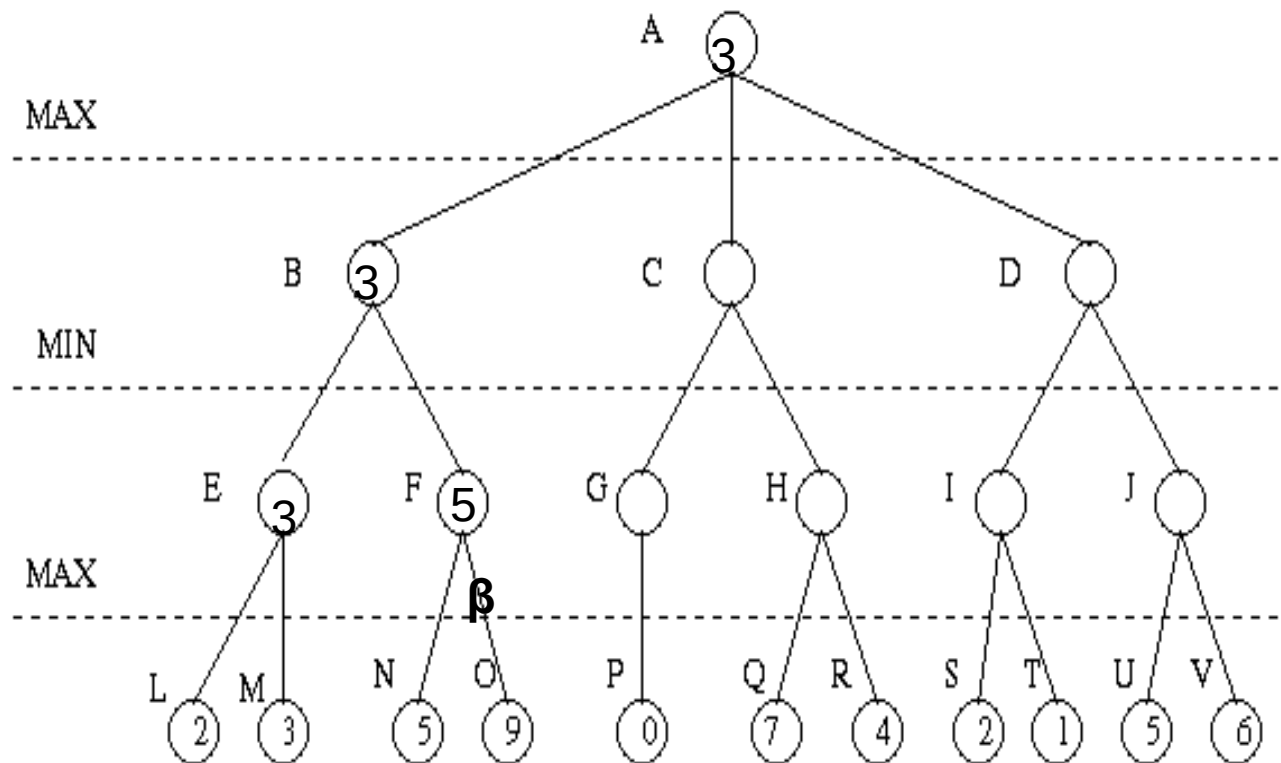
Alpha-Beta Pruning Example

1. Хэрэв beta утга нь MAX өвөг эцгийн alpha утгаас $\leq$ үед MIN зангилаанаас бага зангилааг хайж alpha тайралт хийнэ.
2. Хэрэв alpha утга нь MIN өвөг эцгийн alpha утгаас $\geq$ үед MIN зангилаанаас бага зангилааг хайж beta тайралт хийнэ.



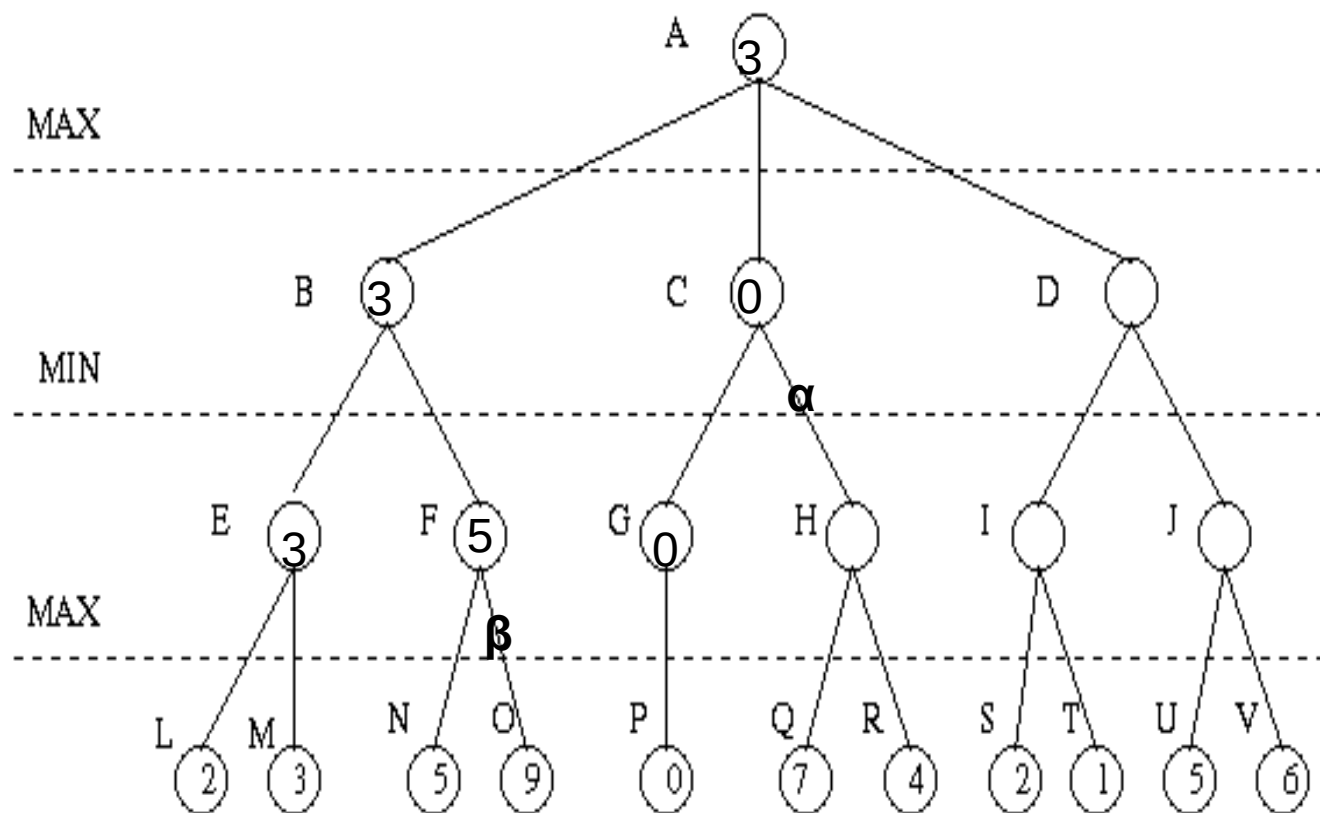
Alpha-Beta Pruning Example

1. Хэрэв beta утга нь MAX өвөг эцгийн alpha утгаас $\leq$ үед MIN зангилаанаас бага зангилааг хайж alpha тайралт хийнэ.
2. Хэрэв alpha утга нь MIN өвөг эцгийн alpha утгаас $\geq$ үед MIN зангилаанаас бага зангилааг хайж beta тайралт хийнэ.



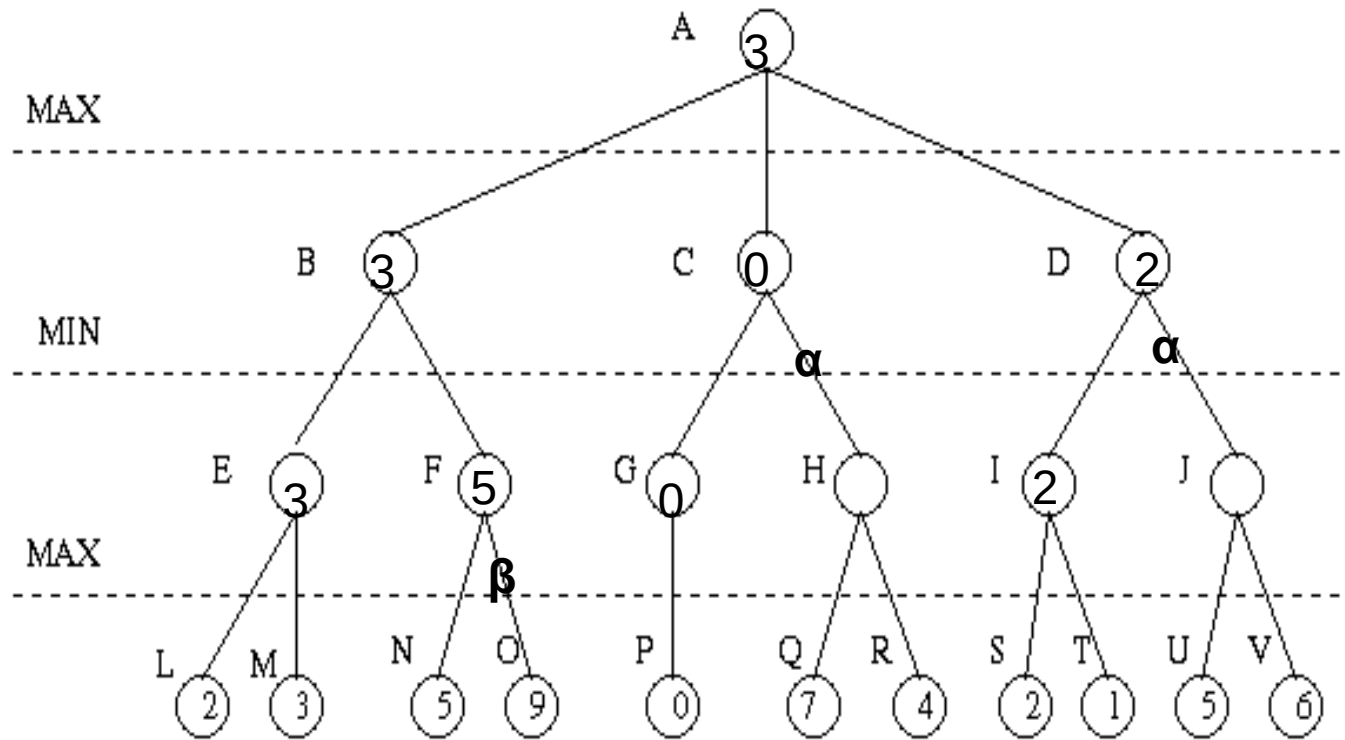
Alpha-Beta Pruning Example

1. Хэрэв beta утга нь MAX өвөг эцгийн alpha утгаас $\leq$ үед MIN зангилаанаас бага зангилааг хайж alpha тайралт хийнэ.
2. Хэрэв alpha утга нь MIN өвөг эцгийн alpha утгаас $\geq$ үед MIN зангилаанаас бага зангилааг хайж beta тайралт хийнэ.

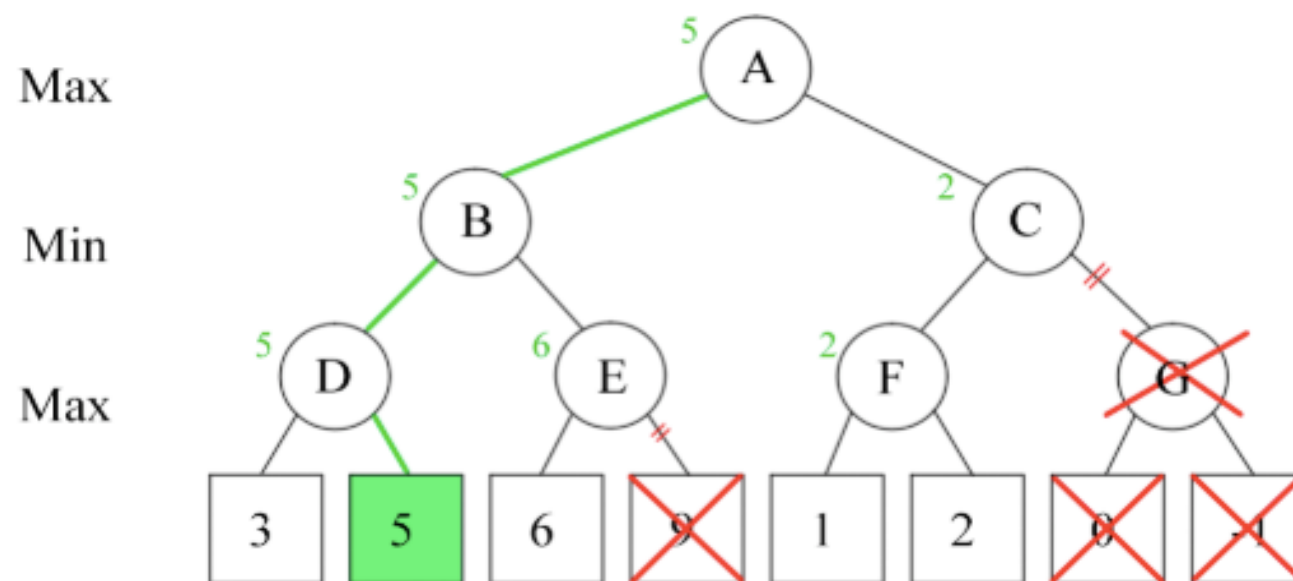


Alpha-Beta Pruning Example

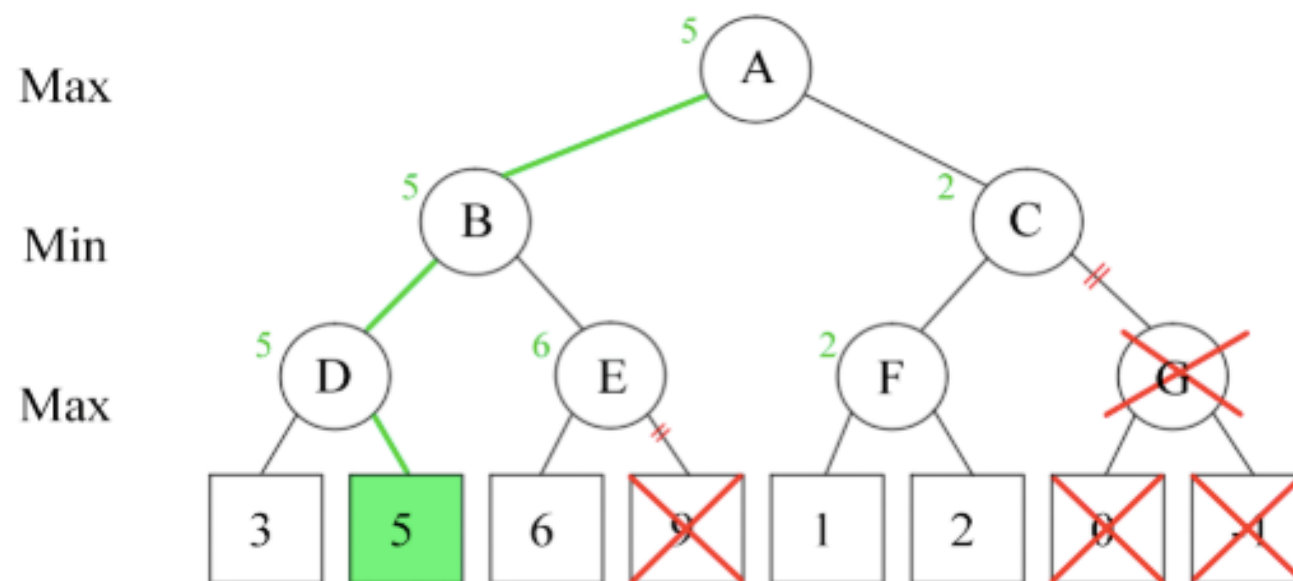
1. Хэрэв beta утга нь MAX өвөг эцгийн alpha утгаас $\leq$ үед MIN зангилаанаас бага зангилааг хайж alpha тайралт хийнэ.
2. Хэрэв alpha утга нь MIN өвөг эцгийн alpha утгаас $\geq$ үед MIN зангилаанаас бага зангилааг хайж beta тайралт хийнэ.



Alpha-Beta Процедур



Alpha-Beta Процедур



α - β pruning дасгал

max

min

max

eval

